

Rapport de stage

Évaluation et développement de méthodes Deep Learning pour l'analyse de données single-cell RNA-seq

Fabien Bidet

Avril 2026

Période de stage : Janvier-Juin 2026

Organisme d'accueil : LaBRI (Laboratoire Bordelais de Recherche en Informatique)

Responsables de stage :

- **Adélaïde Raguin** – Chargée de recherche CNRS, chercheuse au LaBRI (Tutrice de stage)
- **Victoria Bourgeois** – Maître de Conférences, chercheuse au LaBRI, Université de Bordeaux (Encadrante universitaire)
- **Loann Giovannangeli** – Maître de Conférences, chercheur au LaBRI, Université de Bordeaux (Encadrant universitaire)
- **Patricia Thébault** – Professeure des Universités, chercheuse au LaBRI, Université de Bordeaux (Encadrante universitaire)

Table des matières

1	Introduction	1
1.1	Environnement de travail	1
1.2	Introduction du sujet de stage	2
2	État de l’art	4
2.1	Algorithmes de clustering	4
2.1.1	Approches classiques	4
2.1.2	Approches Deep Learning (Apprentissage Profond)	5
2.1.3	Les approches spécifiques à la problématique des cellules rares	6
2.2	Données	7
2.3	Traitements de données	9
2.3.1	Prétraitement initial	9
2.3.2	Effet de batch	9
3	Comparaison de l’existant	10
3.1	Logiciel	11
3.2	Expériences	13
3.2.1	Données et splits	13
3.2.2	Méthodes	14
3.2.3	Résultats	16
4	scRAW	17
4.1	Positionnement et motivation	17
4.2	Pipeline	18
4.3	Fonctions de loss $\mathcal{L}$ et variantes	19
4.4	Recherche d’hyperparamètres	21
4.5	Résultats et discussion	22
4.5.1	Validation sur des jeux non vus pendant l’optimisation	24
5	Travaux complémentaires	26
5.1	Transfert de loss	26
5.2	Induction	27
5.3	Interprétation biologique	27
5.4	Limites méthodologiques	29
	Conclusion	29
	Glossaire et abréviations	31
A	Structure et organigramme du LaBRI	32
B	Détails des jeux de données	34
C	Tableau des métriques d’évaluation	36

D Résultats complets sur les huit jeux de données de l'annexe B	37
E Résultats complets du transfert de loss pondérée $\mathcal{L}_{\text{recon,ponderee}}$	44
F Détails de l'évaluation inductive	46
G Étude d'ablation de scRAW	48
H Hyperparamètres de la configuration de référence scRAW	50
I Figures de recherche d'hyperparamètres scRAW	52
J Screenshots de l'interface graphique SCRBenchmark	55

Table des figures

1	Structure logique et flux de travail de la plateforme de benchmarking SCR-Benchmark.	12
2	Panel de résultats sur le jeu de données Baron.	17
3	Pipeline général de scRAW.	18
4	Distribution des performances sur les huit jeux de données communs pour scRAW et le top 3 de chaque famille de méthodes.	23
5	Matrice de rangs des algorithmes selon les métriques moyennes sur le jeu commun common-8.	24
6	Comparaison entre les algorithmes de base et leur meilleure variante avec loss pondérée scRAW.	26
7	Comparaison inductive de scRAW avec scNAME, scMAE et scDeepCluster sur six jeux de données.	28
8	Analyse biologique compacte de scRAW sur Baron Human Pancreas.	29

Liste des tableaux

1	Synthèse des méthodes de clustering et de détection de populations rares citées dans l'état de l'art.	6
2	Structure conceptuelle d'un jeu de données scRNA-seq.	9
3	Principaux outils de correction de l'effet de batch.	10
4	Composition du jeu de données Baron après filtrage.	13
5	Résultats sur deux jeux de données non vus pendant l'optimisation des hyper-paramètres.	25

1 Introduction

1.1 Environnement de travail

Ce stage de fin d'études s'est déroulé du 4 janvier au 30 juin 2026 au sein du Laboratoire Bordelais de Recherche en Informatique (LaBRI). Le LaBRI est une unité mixte de recherche associée au CNRS (UMR 5800), à l'Université de Bordeaux et à Bordeaux INP, en partenariat avec Inria. J'ai intégré l'équipe Bench to Knowledge and Beyond (BKB), dirigée par Romain Bourqui (responsable d'équipe) et Patricia Thébault (adjointe). Cette équipe, rattachée au département Systèmes et Données (SeD), travaille sur la gestion, l'analyse et l'exploration de données complexes, avec des applications importantes en bioinformatique, en santé et en visualisation de données. Afin de mieux appréhender la structure dans laquelle j'ai évolué, l'organisation détaillée du laboratoire ainsi que son organigramme structurel sont présentés en Annexe [A](#).

Équipe d'encadrement

Au cours de ce stage, j'ai été encadré par une équipe aux expertises complémentaires :

- **Victoria Bourgeois**, enseignante-chercheuse au LaBRI, experte dans l'application de l'apprentissage profond au domaine de la santé ;
- **Loann Giovannangeli**, enseignant-chercheur au LaBRI, en visualisation de l'information et intelligence artificielle ;
- **Patricia Thébault**, enseignante-chercheuse au LaBRI, en sciences des données omiques et réseaux biologiques.

Cadre de travail

J'ai effectué mes travaux dans un bureau partagé avec des doctorants et d'autres stagiaires. Ce contexte s'est révélé particulièrement enrichissant, facilitant les échanges de connaissances au quotidien et me permettant de mieux cerner les enjeux d'une thèse et de la vie de jeune chercheur(se). Ces échanges m'ont aussi donné l'occasion de présenter régulièrement mes résultats et de préparer progressivement ma soutenance.

Par ailleurs, j'ai pu assister aux séminaires hebdomadaires organisés par les doctorant(e)s de l'équipe BKB, durant lesquels des chercheurs, doctorants et stagiaires, présentaient leurs travaux en bioinformatique. J'ai par ailleurs eu l'occasion lors de ces séminaires de présenter à deux reprises mes travaux, choses qui m'a était importante pour préparer ma soutenance finale. J'ai également eu l'opportunité de participer aux séminaires d'autres équipes, notamment celles spécialisées en intelligence artificielle. Ces événements ont constitué une source d'apprentissage scientifique de pointe, idéale pour mon profil situé à l'interface entre la bioinformatique et l'intelligence artificielle. D'un point de vue technique, j'ai bénéficié d'un accès distant (via SSH) à un serveur de calcul interne au LaBRI pour réaliser mes expérimentations.

Suivi et méthodologie

L'avancement de mes travaux a été rythmé par des réunions régulières. Une réunion hebdomadaire avec mes encadrants permettait de faire le point sur l'avancement scientifique, les difficultés rencontrées et les priorités expérimentales. Des échanges plus informels complétaient ce suivi au quotidien. De plus, des points bimensuels avec Elodie Darbot et Cyril Dourthe, deux bioinformaticien(ne)s en lien avec le BRIC, ainsi qu'avec plusieurs stagiaires, ont permis de présenter l'avancement du stage, de discuter les choix méthodologiques et de bénéficier de leurs expertises complémentaires pour les questions bioinformatiques. Enfin, j'ai pu compter sur la disponibilité de ma tutrice de stage Adélaïde Raguin, chercheuse au LaBRI, pour faire le point sur mon avancement et répondre à mes interrogations tout au long de cette période.

1.2 Introduction du sujet de stage

Le séquençage de l'ARN à l'échelle de la cellule unique (*single-cell RNA sequencing*, scRNA-seq) a profondément transformé l'analyse des tissus biologiques. Contrairement aux approches transcriptomiques classiques, comme l'analyse bulk-ARN, qui mesurent une expression moyenne sur un ensemble de cellules, le scRNA-seq permet d'observer les profils d'expression cellule par cellule. Il devient alors possible d'étudier l'hétérogénéité interne d'un tissu, d'identifier différents types ou états cellulaires, et de mettre en évidence des populations minoritaires qui auraient été masquées dans une mesure globale [1, 2, 3, 4, 5]. Par exemple, cette technologie a permis d'identifier des types cellulaires rares dans l'intestin [1] et de cartographier le développement des cellules germinales humaines [2]. Elle est également cruciale pour étudier l'hétérogénéité tumorale [3], la formation de la mémoire dans le cerveau [4] ou encore les altérations cellulaires associées à la maladie d'Alzheimer [5].

Cette richesse d'information s'accompagne cependant de difficultés importantes. Les données scRNA-seq sont de très grande dimension, car chaque cellule est décrite par l'expression de plusieurs milliers de gènes. Elles sont également bruitées, très creuses, sensibles aux événements de non-détection (effet dropout) et souvent affectées par des effets de batch liés au protocole expérimental, au laboratoire ou à la plateforme de séquençage [6]. Avant toute interprétation biologique, il est donc nécessaire de construire une représentation plus compacte et plus robuste des cellules.

Dans ce contexte, le clustering constitue une étape centrale des analyses scRNA-seq. Il s'agit d'une forme de classification non supervisée : les cellules sont regroupées à partir de profils d'expression similaires afin de retrouver des types cellulaires, des sous-types ou des états fonctionnels. Des pipelines classiques, comme ceux fondés sur une réduction de dimension suivie d'un clustering, sont aujourd'hui largement utilisés dans les outils bioinformatiques standards [7, 8]. Plus récemment, les méthodes d'apprentissage profond ont cherché à apprendre des espaces latents plus adaptés à la complexité des données, notamment à l'aide d'autoencodeurs ou de modèles probabilistes [9].

Un des enjeux abordés pendant ce stage concerne plus spécifiquement les **populations cellulaires rares**. En biologie, ces populations peuvent correspondre à des cellules immunitaires minoritaires, à un sous-type tumoral, à une population pathologique, à un état transitoire de différenciation ou encore à des cellules spécifiques d'un tissu. Leur faible proportion ne signifie

donc pas qu'elles sont secondaires : au contraire, elles peuvent porter un signal biologique essentiel. Pourtant, du point de vue algorithmique, elles sont particulièrement fragiles. Comme elles représentent peu d'observations, elles contribuent faiblement à l'apprentissage des modèles et peuvent facilement être absorbées et confondues par une population majoritaire ou par un type cellulaire proche. Un bon score moyen dans une métrique généraliste peut alors masquer un échec important : la disparition d'une population biologiquement pertinente.

L'enjeu de ce mémoire est alors de développer une approche capable d'obtenir un clustering global précis tout en préservant les cellules rares, y compris lorsque les jeux de données comportent un bruit technique important ou des effets de batch. Cette approche doit aussi rester utilisable au-delà des données qui ont servi à la construire. Cette idée recouvre deux niveaux complémentaires : d'une part, des hyperparamètres fixés doivent rester pertinents sur des jeux de données qui n'ont pas participé à leur optimisation ; d'autre part, dans un cadre inductif, un modèle déjà entraîné doit pouvoir être appliqué à de nouvelles cellules ou à un nouveau patient sans être entièrement réentraîné. Cette seconde logique est importante en pratique, car l'entraînement peut être coûteux et peu souhaitable à répéter pour chaque nouvel échantillon. Elle faciliterait la comparaison de nouveaux patients avec des cohortes déjà analysées, tout en limitant le risque de manquer une sous-population minoritaire biologiquement ou cliniquement pertinente. Pour cela, il existe d'une part des approches transductives, qui travaillent uniquement avec les données disponibles, et d'autre part des approches inductives, qui peuvent généraliser à de nouvelles données.

Mon stage s'est inscrit dans cette problématique à l'interface entre bioinformatique, apprentissage automatique et évaluation expérimentale. Le travail réalisé s'est articulé autour de trois contributions principales. La première a consisté à développer et structurer **SCRBenchmark**, une interface web interactive destinée à comparer différentes méthodes de clustering scRNA-seq sur plusieurs jeux de données. La seconde contribution a été le développement de **scRAW**, une méthode visant à apprendre une représentation latente en donnant davantage d'importance aux cellules rares pendant l'apprentissage. Ces travaux ont donné lieu à une soumission acceptée en poster à JOBIM 2026, intitulée *scRAW : Representation learning for rare cell population identification*. Enfin, des travaux complémentaires sur l'approche développée ont exploré l'intégration de cette logique de pondération sur d'autres modèles, les premiers tests d'un comportement inductif de scRAW et l'interprétation biologique des populations identifiées par notre modèle.

La question directrice du mémoire peut ainsi se formuler de la manière suivante : comment construire, évaluer et améliorer un pipeline de clustering scRNA-seq capable de mieux préserver les populations cellulaires rares, tout en restant reproductible et utilisable sur de nouvelles données ? Pour y répondre, les notions nécessaires à la compréhension des données scRNA-seq et des méthodes de clustering sont d'abord introduites. Le protocole de comparaison mis en place est ensuite décrit, avant de détailler la conception et l'évaluation de scRAW. Enfin, les extensions explorées pendant le stage sont discutées, notamment le transfert de loss, l'induction et les pistes d'interprétation biologique.

2 État de l’art

L’objectif de cette section est de présenter les bases nécessaires à la compréhension du rapport pour mieux comprendre mes contributions et où elles se situent par rapport à la recherche actuelle. Deux notions sont particulièrement importantes pour la suite : l’effet de batch, qui désigne une variation technique pouvant séparer les cellules selon leur origine expérimentale plutôt que selon leur biologie, et les autoencodeurs, qui apprennent une représentation compacte des cellules en reconstruisant leur profil d’expression. Les principales familles d’algorithmes de clustering sont d’abord présentées, avant de décrire les jeux de données utilisés et les traitements nécessaires pour les préparer.

2.1 Algorithmes de clustering

Le clustering est une tâche non supervisée : les cellules sont regroupées à partir de leurs profils d’expression sans fournir directement au modèle les annotations biologiques attendues. L’objectif est de produire des groupes cohérents avec les types cellulaires, les sous-types ou les états fonctionnels présents dans le tissu. En scRNA-seq, ce regroupement est généralement réalisé après une réduction de dimension, car chaque cellule est décrite par plusieurs milliers de gènes.

2.1.1 Approches classiques

Les approches classiques reposent le plus souvent sur une représentation de dimension réduite, par exemple une analyse en composantes principales (PCA), qui résume l’expression de milliers de gènes en un plus petit nombre d’axes. Le clustering peut ensuite être réalisé selon plusieurs familles d’algorithmes :

- **Méthodes basées sur les centroïdes** : K-Means [10] cherche à représenter chaque groupe par un centre moyen, ou centroïde. Chaque cellule est affectée au centroïde le plus proche, puis les centroïdes sont mis à jour de manière itérative. Ces méthodes sont rapides et simples à interpréter, mais elles supposent souvent des groupes relativement compacts et nécessitent de fixer le nombre de clusters.
- **Méthodes basées sur les graphes** : Louvain [11] et Leiden [12] construisent un graphe de voisinage dans lequel les cellules sont reliées à leurs plus proches voisines. Le clustering consiste alors à détecter des communautés fortement connectées dans ce graphe. Cette famille est très utilisée en single-cell, notamment dans Scanpy [7], car elle s’adapte bien à des structures non linéaires.
- **Méthodes basées sur la densité** : HDBSCAN [13] cherche des régions denses dans l’espace de représentation et peut laisser certains points comme bruit. Cette propriété est utile lorsque les clusters n’ont pas tous la même forme ou la même densité, mais les petits groupes peuvent rester difficiles à distinguer si leur signal est trop faible.

Ces méthodes sont efficaces et largement utilisées. Toutefois, elles dépendent fortement du prétraitement et de la représentation d’entrée. Une population très petite peut ne pas

former sa propre communauté dans un graphe, être absorbée par le centroïde d’une population voisine ou être considérée comme du bruit par une méthode de densité.

2.1.2 Approches Deep Learning (Apprentissage Profond)

Les méthodes basées sur le deep learning apprennent une représentation latente des cellules au lieu de s’appuyer uniquement sur une projection linéaire comme la PCA. Elles sont souvent fondées sur des autoencodeurs. Un autoencodeur est un réseau de neurones composé de deux parties : un encodeur et un décodeur. L’encodeur transforme le profil d’expression d’une cellule, initialement décrit par des milliers de gènes, en un vecteur plus compact appelé espace latent. Le décodeur tente ensuite de reconstruire le profil d’expression initial à partir de ce vecteur latent. Si la reconstruction est correcte, cela signifie que l’espace latent conserve une partie importante de l’information biologique utile.

Dans ce cadre, l’apprentissage consiste à minimiser une *loss* notée $\mathcal{L}$. Cette *loss* pénalise le modèle lorsqu’il reconstruit mal les données, regroupe mal les cellules ou conserve trop d’information technique. La *loss* totale $\mathcal{L}_{\text{total}}$ peut être vue comme une somme pondérée de plusieurs objectifs, de la même façon qu’une pénalisation de type L2 ajoute un terme d’optimisation à l’objectif principal.

Pour bien comprendre, regardons sur quoi se base la *loss* totale $\mathcal{L}_{\text{total}}$ (*total loss*) de différents modèles de l’état de l’art :

- **La *loss* de reconstruction $\mathcal{L}_{\text{recon}}$** : Des modèles comme scVI [9] ou scMAE [14] cherchent avant tout à bien reconstruire le profil d’expression initial de la cellule. Si le modèle compresse mal l’information et n’arrive pas à deviner les gènes de départ, cette *loss* augmente.
- **La *loss* de clustering $\mathcal{L}_{\text{cluster}}$** : Des méthodes comme DESC [15], scDeepCluster [16] ou scNAME [17] ajoutent une contrainte supplémentaire. Elles mesurent les distances entre les cellules et forcent le modèle à regrouper les cellules qui se ressemblent de plus en plus au fil du temps, souvent en minimisant une mesure de différence appelée divergence KL.
- **La *loss* de correction de batch $\mathcal{L}_{\text{batch}}$** : Certaines architectures adversariales, comme D2AN [18] ou ABC [19], ajoutent un terme adverse (via un discriminateur) qui cherche à rendre le batch difficile à prédire à partir de l’espace latent, forçant ainsi l’alignement des données durant l’apprentissage.
- **Les autres *loss* ou régularisations** : D’autres modèles ajoutent des contraintes de voisinage, de contraste ou de graphe afin de stabiliser l’espace latent. Ces termes ne sont pas toujours détaillés ici, mais ils reposent sur la même idée générale : orienter l’apprentissage vers une représentation plus utile pour le clustering.

Le problème avec les populations rares vient de la manière dont cette *loss* totale $\mathcal{L}_{\text{total}}$ est calculée. Le modèle cherche à minimiser cette *loss* globale sur l’ensemble des données. Les populations majoritaires, qui contiennent des milliers de cellules, ont un poids important dans ce calcul. À l’inverse, une erreur sur un type cellulaire rare contenant seulement quelques dizaines de cellules modifie très peu $\mathcal{L}_{\text{total}}$. Le réseau de neurones peut donc apprendre une représentation globalement performante tout en négligeant les petites populations.

Famille	Méthode	Rôle	Framework	Objectif / loss
Approches classiques	PCA [20]	Réduction de dimension	Projection linéaire	Maximisation de la variance expliquée
	K-Means [10]	Clustering	Centroïdes	Minimisation de l'inertie intra-cluster
	Louvain [11]	Clustering	Graphe de voisinage	Maximisation de la modularité
	Leiden [12]	Clustering	Graphe de voisinage	Modularité avec communautés mieux connectées
	HDBSCAN [13]	Clustering	Densité hiérarchique	Identification de groupes denses et de points bruit
Deep Learning	scVI [9]	Réduction de dimension	VAE probabiliste	Reconstruction NB/ZINB + divergence KL
	scMAE [14]	Réduction de dimension	Masked AE	Reconstruction de gènes masqués
	DESC [15]	Les deux	AE + clustering	Reconstruction + clustering par divergence KL
	scDeepCluster [16]	Les deux	AE ZINB	Reconstruction ZINB + clustering par divergence KL
	scNAME [17]	Les deux	Self-supervised	Reconstruction + clustering + regroupement des cellules voisines
	scCDCG [21]	Les deux	AE + graph embedding	Reconstruction + clustering par structure de graphe et alignement
Cellules rares	GiniClust [22]	Clustering	Indice de Gini	Détection de gènes marqueurs atypiques
	CellSIUS [23]	Clustering	Signatures locales	Identification de sous-populations minoritaires
	scAIDE [24]	Les deux	AE	Débruitage + clustering orienté populations rares
	DeepScena [25]	Les deux	Self-supervised	Raffinement des clusters rares
	scCAD [26]	Clustering	Détection d'anomalies	Décomposition de clusters pour isoler les types rares

TABLE 1 – Synthèse des méthodes de clustering et de détection de populations rares citées dans l'état de l'art. AE : autoencodeur ; VAE : autoencodeur variationnel ; KL : divergence de Kullback-Leibler ; NB/ZINB : lois binomiale négative et binomiale négative à inflation de zéros.

2.1.3 Les approches spécifiques à la problématique des cellules rares

Face à ces limites, l'identification des populations cellulaires minoritaires est devenue un enjeu central dans l'analyse des données single-cell RNA-seq. Plusieurs algorithmes ont ainsi été développés pour mieux détecter ces sous-populations rares. Tous s'appuient, d'une manière ou d'une autre, sur les profils d'expression des gènes, mais ils ne les exploitent pas de la même façon.

Certaines méthodes cherchent directement des gènes caractéristiques des cellules rares. Par exemple, **GiniClust** [22] utilise l'indice de Gini pour repérer des gènes exprimés dans seulement un petit nombre de cellules. Ces gènes peuvent alors signaler l'existence d'une population rare qui serait difficile à voir avec un clustering classique. Dans une logique proche, **CellSIUS** [23] part de groupes cellulaires déjà formés, puis recherche à l'intérieur de ces groupes des signatures d'expression particulières permettant d'identifier des sous-populations

minoritaires.

D'autres méthodes cherchent plutôt à améliorer la représentation globale des cellules avant de les regrouper. C'est le cas de **scAIDE** [24], qui utilise un autoencodeur pour réduire le bruit des données et produire une représentation plus claire des cellules. Une fois cette représentation obtenue, scAIDE applique une méthode de clustering conçue pour limiter le biais en faveur des grandes populations cellulaires. L'objectif est donc de mieux séparer les petits groupes de cellules, y compris lorsqu'ils sont très peu représentés dans le jeu de données.

Dans une approche proche, mais davantage hiérarchique, **DeepScena** [25] commence par identifier de grands groupes cellulaires, puis réanalyse progressivement certains de ces groupes afin de faire apparaître des sous-populations plus fines. À chaque niveau, les gènes les plus informatifs peuvent être recalculés dans le sous-groupe étudié, ce qui permet de mieux détecter des différences qui étaient invisibles à l'échelle globale. DeepScena ne donne donc pas directement plus de poids aux cellules rares dans sa loss ; il les révèle plutôt grâce à un raffinement progressif du clustering et à une meilleure modélisation des données single-cell.

Enfin, des méthodes plus récentes comme **scCAD** [26] abordent la question sous l'angle de la détection d'anomalies. L'idée est d'identifier les cellules dont le profil d'expression s'éloigne de celui des populations majoritaires, puis de décomposer progressivement les clusters afin d'isoler ces groupes atypiques.

Ces méthodes reposent donc sur une idée commune : un clustering global classique tend à favoriser les grandes populations cellulaires et peut masquer les cellules rares. L'approche développée pendant ce stage, **scRAW**, s'inscrit dans cette continuité tout en proposant une stratégie différente. Plutôt que d'identifier les populations rares uniquement après la construction des clusters, nous cherchons à intégrer cette contrainte directement pendant l'apprentissage du modèle. Concrètement, scRAW attribue un poids plus important aux cellules isolées ou faiblement représentées dans le calcul de la loss de reconstruction, afin qu'elles influencent davantage la construction de l'espace latent. L'objectif est ainsi de produire une représentation latente plus informative, dans laquelle les populations rares ne sont pas absorbées par les grands groupes cellulaires, tout en étant mises en évidence tout au long de l'entraînement du modèle.

2.2 Données

Pour évaluer la robustesse d'une méthode de clustering, il est nécessaire de la tester sur des données variées. Dans le cadre de ce stage, nous nous sommes appuyés sur la collection de huit jeux de données du sous-ensemble commun (détaillés en Annexe B).

Ces données proviennent de différents organes et espèces : pancréas humain, cerveau, rate, sang (PBMC), testicules, foie, moelle osseuse, ou encore rétine de macaque. Elles couvrent plusieurs difficultés expérimentales :

- **La taille** : Cela va de petits jeux de données d'environ 2 700 à 3 000 cellules (comme *Paul15 bone marrow* ou *Tabula Muris liver*) jusqu'à de très gros ensembles de cellules dépassant les 30 000 cellules (comme la rétine de macaque ou les PBMC).
- **Les effets de batch** : La plupart des jeux de données sont composés de plusieurs batchs expérimentaux. Ces batchs peuvent représenter des donneurs, des individus ou

des conditions différentes. Par exemple, les 4 batchs du pancréas humain de Baron correspondent à quatre donneurs distincts, les 16 batchs de Kang PBMC combinent différents donneurs et conditions (cellules stimulées à l'interféron et contrôles), et les 30 batchs de la rétine de macaque proviennent d'échantillons prélevés sur différents individus.

- **Les populations rares :** C'est le cœur de notre problématique. Presque tous nos jeux de données contiennent des types cellulaires dits "rares" (représentant moins de 5 % des cellules) ou "ultra-rares" (moins de 1 %). Ces seuils ont été choisis de manière empirique à partir du jeu de données Baron, où la distribution des types cellulaires fait apparaître trois groupes lisibles : les classes fréquentes au-dessus de 5 %, les classes rares en dessous de 5 % et les classes ultra-rares en dessous de 1 %. Par exemple, dans les données du pancréas humain de Baron, les cellules de Schwann ne représentent que 0,15 % du total, tandis que certaines populations immunitaires comme les mastocytes ou les lymphocytes T ne représentent respectivement que 0,29 % et 0,08 % des cellules.

L'utilisation d'annotations pré-existantes (la "vérité terrain") nous permet d'évaluer objectivement si les algorithmes parviennent à retrouver ces populations ou si le signal est noyé. Le tableau 6 (disponible en Annexe B) résume l'ensemble des huit jeux de données de référence retenus pour notre étude, détaillant pour chacun d'eux l'espèce d'origine, le nombre de cellules et de gènes, le nombre de batchs, la quantité de populations cellulaires rares, ainsi que leur effet de batch (PCR sur PCA) et leur déséquilibre de classe (indice de Gini).

Ces huit jeux de données ont servi à construire et comparer les configurations principales. Pour contrôler plus directement la capacité de scRAW à conserver de bonnes performances sans réoptimiser ses hyperparamètres, deux jeux supplémentaires ont ensuite été réservés à une validation externe : *Human Pancreas* et *SCIB Mouse Pancreas 1*. Ils n'ont pas été utilisés pendant la recherche d'hyperparamètres de scRAW et sont décrits séparément en Annexe B (Tableau 8).

Les données de séquençage d'ARN sur cellule unique (scRNA-seq) manipulées sont généralement structurées sous forme de matrices d'expression accompagnées de métadonnées. Par convention (notamment dans le format `AnnData` utilisé par notre pipeline), ces informations sont organisées de la façon suivante :

- **La matrice d'expression (X) :** De dimension $N \times G$ (où N est le nombre de cellules et G le nombre de gènes). Chaque entrée représente le niveau d'expression (par exemple le nombre de comptages d'U.M.I., pour *Unique Molecular Identifiers*) d'un gène spécifique dans une cellule donnée. Cette matrice est généralement très creuse (majoritairement remplie de zéros).
- **Les métadonnées cellulaires (*observations* ou *obs*) :** Une table de dimension $N \times C$ alignée sur les lignes de la matrice d'expression. Elle attribue à chaque cellule des informations de contexte (comme le batch expérimental d'origine ou l'annotation biologique) réparties sur C colonnes.
- **Les métadonnées des gènes (*variables* ou *var*) :** Une table de dimension $G \times V$ alignée sur les colonnes de la matrice d'expression, décrivant les caractéristiques des

gènes (nom du gène, niveau de variabilité globale, etc.) réparties sur V colonnes.

Le tableau 2 illustre un exemple conceptuel de cet alignement entre la matrice d'expression et les métadonnées cellulaires.

Identifiant Cellule	Matrice d'expression (X)			Métadonnées des cellules (obs)	
	Gène A	Gène B	Gène C	Batch	Type cellulaire
Cellule_1	0	14	0	Batch_A	Cellule Beta
Cellule_2	3	0	1	Batch_A	Cellule Alpha
Cellule_3	0	0	0	Batch_B	Cellule de Schwann
Cellule_4	24	1	0	Batch_B	Cellule Beta
Cellule_5	0	0	8	Batch_C	Lymphocyte T

TABLE 2 – Structure conceptuelle d'un jeu de données scRNA-seq. La matrice d'expression à gauche est alignée ligne à ligne (par cellule) avec la table des métadonnées cellulaires à droite.

2.3 Traitements de données

Les données brutes issues du séquençage scRNA-seq sont extrêmement bruitées et pleines de zéros (des gènes non détectés). Il est donc indispensable de les nettoyer et de les préparer.

2.3.1 Prétraitement initial

Le traitement standard [6] consiste d'abord à filtrer les cellules de mauvaise qualité et les gènes très peu exprimés. Ensuite, on normalise les données pour que toutes les cellules soient comparables, puis on applique une transformation mathématique (le logarithme) pour atténuer les écarts extrêmes. Enfin, on ne conserve généralement que les gènes dits "hautement variables" (HVG), c'est-à-dire ceux qui différencient le mieux les cellules entre elles. Bien que ces étapes soient standardisées, elles présentent un risque pour les cellules rares : un filtrage trop agressif peut effacer les quelques gènes qui caractérisaient spécifiquement une toute petite population.

2.3.2 Effet de batch

Un batch correspond à un groupe de cellules ayant partagé une même condition technique ou expérimentale : journée de manipulation, laboratoire, protocole de préparation, plateforme de séquençage, donneur, technologie ou plaque expérimentale. L'effet de batch désigne la variation introduite par ces conditions, indépendamment des différences biologiques recherchées. Dans un cas idéal, deux cellules du même type cellulaire devraient être proches même si elles proviennent de batches différents. En pratique, un effet de batch fort peut conduire les cellules à se regrouper par origine expérimentale plutôt que par type cellulaire.

Ce biais complique directement le clustering. Une méthode peut séparer deux batches d'un même type cellulaire comme s'il s'agissait de deux populations biologiques différentes, ou au contraire masquer une population rare si celle-ci est fortement associée à un seul batch. Ces méthodes peuvent être associées à la tâche de préparation des données, ou directement au cœur

de la méthode d'apprentissage. Il existe donc des méthodes dédiées pour corriger ce problème (présentées dans le tableau 3), comme ComBat [27], Harmony [28], Scanorama [29], ou scVI [9]. L'objectif est d'aligner les différents batchs pour réduire la signature technique. Cependant, la difficulté est de ne pas "sur-corriger" : en voulant effacer les différences techniques, le modèle peut aussi gommer de vraies différences biologiques, en particulier celles des populations rares qui pourraient n'apparaître que dans un seul batch.

Une autre stratégie consiste à corriger l'effet de batch directement pendant l'apprentissage, par exemple en utilisant le principe des réseaux adversariaux DANN (*Domain-Adversarial Neural Networks*) [30]. L'idée est de pénaliser le modèle si le batch d'origine reste facile à deviner à partir de l'espace latent, ce qui l'incite à supprimer cette information technique. Cette approche a été adaptée aux données biologiques, notamment avec scDGN pour aligner des expériences différentes [31], ou encore avec D2AN [18] et ABC [19] pour retirer l'effet de batch. Comme pour les autres méthodes, le risque est de faire disparaître de réelles différences biologiques si celles-ci sont confondues avec l'effet de batch.

Outil	Méthodologie	Principe de correction
ComBat [27]	Modèle bayésien empirique	Ajustement des moyennes et variances des gènes par batch
DANN [30]	Réseau de neurones adversarial	Inversion de gradient pour masquer l'information de batch dans l'espace latent
Harmony [28]	Soft K-Means itératif	Alignement des cellules similaires de batchs différents en PCA
Scanorama [29]	Mutual Nearest Neighbors (MNN)	Identification de populations cellulaires partagées entre les batchs pour guider l'alignement
scVI [9]	Autoencodeur Variationnel (VAE)	Modélisation probabiliste du bruit et des batchs dans l'espace latent

TABLE 3 – Principaux outils de correction de l'effet de batch.

3 Comparaison de l'existant

Nos travaux ont débuté par une comparaison des méthodes de l'état de l'art traitant des données de séquençage d'ARN à cellule unique (scRNA-seq). À ce stade, nous souhaitions étudier des approches généralistes, qui ne traitaient pas explicitement l'effet de batch ni le déséquilibre des populations cellulaires. Cette première comparaison visait aussi à identifier leurs limites lorsque l'évaluation porte sur des classes rares ou sur des données non vues. Pour ce faire, nous nous sommes en partie appuyés sur le protocole et les sélections du benchmark scCluBench [32], dont nous avons retenu plusieurs méthodes de deep learning basées sur des autoencodeurs ou sur des graphes.

Le protocole de comparaison devait permettre de modifier les hyperparamètres des méthodes tout en contrôlant les étapes de prétraitement. De plus, la quantité de tests différents et de résultats disponibles m'a conduit à développer une solution logicielle ergonomique, SCR-Benchmark, afin d'automatiser la mise en place des expériences et de simplifier l'exploration des résultats.

3.1 Logiciel

Développée en Python, la plateforme SCRBenchmark s'appuie sur la bibliothèque Streamlit pour proposer une interface utilisateur graphique (GUI) interactive, tout en offrant une interface en ligne de commande (CLI) pour l'exécution sur serveur de calcul. Elle a été conçue pour rendre les expériences de comparaison plus simples à construire, plus faciles à reproduire et plus lisibles pour des utilisateurs qui ne travaillent pas nécessairement dans le code au quotidien. Le logiciel regroupe dans un même environnement le chargement des données, le choix du protocole expérimental, le prétraitement, la sélection des méthodes, l'exécution des analyses et l'exploration des résultats. Sa structure logique est résumée dans la figure 1.

Le premier choix important a été de séparer l'interface utilisée pour configurer les expériences du moteur chargé d'exécuter les calculs. L'interface web guide l'utilisateur étape par étape, tandis que la ligne de commande permet de relancer la même analyse sur un serveur ou dans un environnement de calcul plus adapté aux expériences longues. Les neuf écrans principaux de l'interface, consultables en Annexe J, couvrent l'ensemble du parcours : importation des données, visualisation préliminaire, choix du type d'expérience, prétraitement, sélection des algorithmes, génération de la commande d'exécution, suivi du calcul, consultation des métriques et visualisation des UMAP finales.

SCRBenchmark propose ensuite deux modes d'analyse. Le mode standard utilise toutes les cellules ensemble et sert à observer directement le comportement d'une méthode sur un jeu de données. Le mode benchmark sépare au contraire les cellules en ensembles d'entraînement, de validation et de test, afin d'évaluer les méthodes dans des conditions plus contrôlées, notamment lorsqu'il s'agit de tester leur comportement sur des cellules ou des batchs non vus pendant l'entraînement.

L'intérêt principal du logiciel est d'imposer un cadre commun à des méthodes très différentes. Le même pipeline de prétraitement peut être appliqué à toutes les méthodes : filtrage, normalisation, sélection des gènes les plus informatifs et, si nécessaire, correction de l'effet de batch. De la même manière, le moteur d'exécution attend de chaque algorithme les mêmes types de sorties : groupes de cellules prédits, représentation numérique des cellules lorsque la méthode en produit une, paramètres utilisés et temps d'exécution. Ce processus commun permet de s'assurer que les résultats seront issus du fonctionnement des algorithmes et non des données en entrée.

Enfin, SCRBenchmark sauvegarde les éléments nécessaires à l'analyse et à la reproductibilité : scores, labels prédits, embeddings, figures, paramètres et logs. Les visualisations générées automatiquement, comme les UMAP, les matrices de confusion ou les figures comparant entraînement et test, permettent d'explorer rapidement les résultats sans réécrire du code pour chaque expérience. L'outil a donc servi à la fois à produire les expériences et à en analyser les

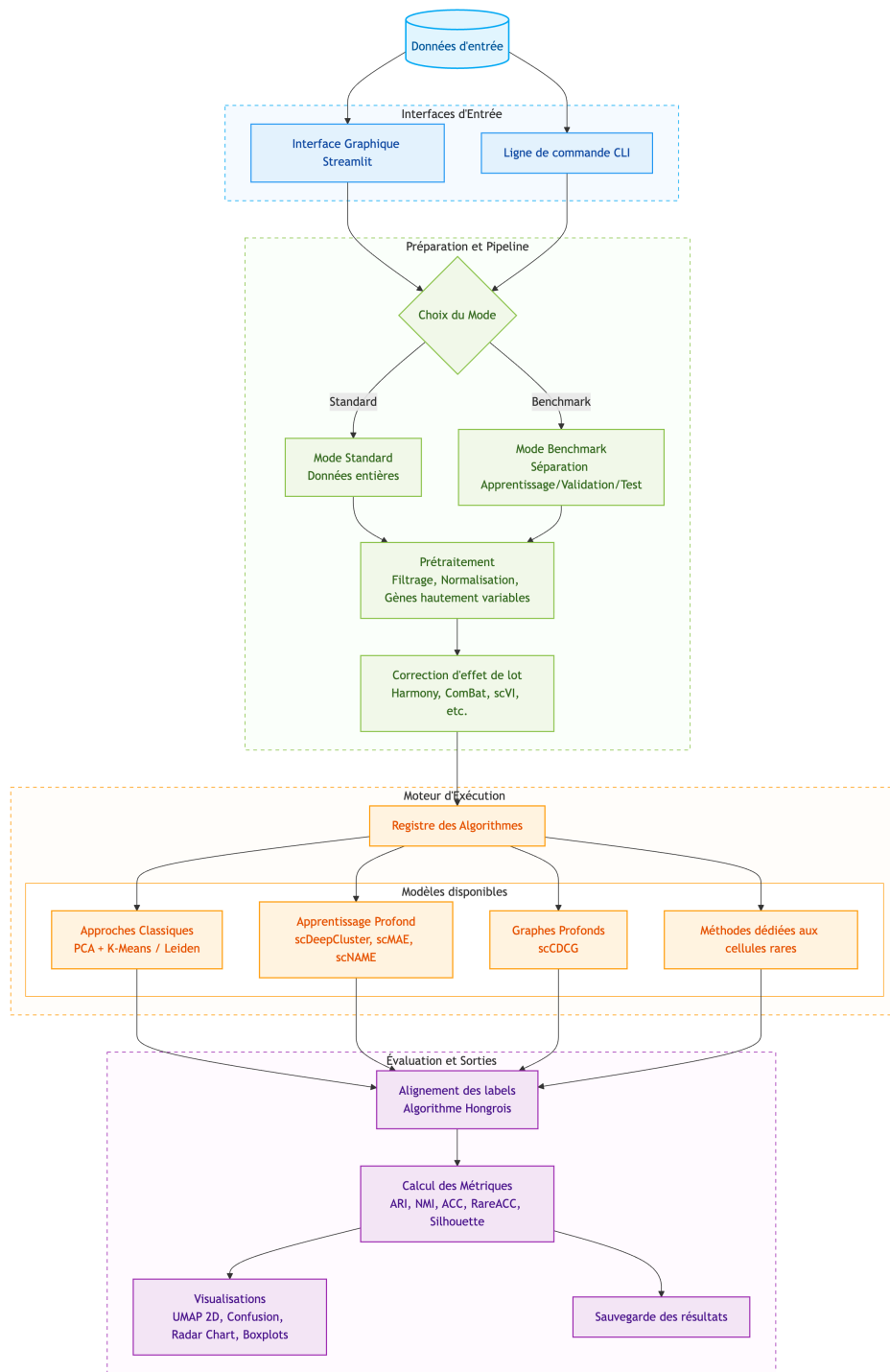


FIGURE 1 – Structure logique et flux de travail de la plateforme de benchmarking SCRBenchmark.

sorties de manière cohérente.

3.2 Expériences

3.2.1 Données et splits

Nous avons retenu le jeu de données Baron [33] pour les expériences présentées ici. Ce choix permettait de réunir dans un même cadre les deux difficultés au centre du mémoire : un effet de batch lié aux donneurs et un fort déséquilibre entre types cellulaires. Le jeu de données regroupe quatre donneurs humains, notés *human1* à *human4*, et constitue un standard de référence largement utilisé pour évaluer les méthodes de clustering en scRNA-seq. Les publications originales de plusieurs méthodes d'apprentissage profond comme scDeepCluster [16], scNAME [17], scMAE [14] et scCDCG [21], ainsi que des approches dédiées aux populations rares telles que CellSIUS [23], scAIDE [24], DeepScena [25], scCAD [26] et le benchmark scCluBench [32], s'appuient sur ce type de données de référence pour valider leurs performances.

Après filtrage, les expériences sur Baron portent sur 8 569 cellules réparties en quatorze types cellulaires. Le tableau 4 montre que les cellules *alpha* et *beta* représentent à elles seules plus de la moitié du jeu de données, tandis que plusieurs populations d'intérêt biologique, comme les *macrophage*, *mast*, *schwann* ou *t_cell*, représentent moins de 1 % des cellules. Ce jeu de données est donc particulièrement adapté pour tester si un algorithme conserve les petites populations au lieu d'optimiser uniquement les grandes classes.

Type cellulaire	Cellules	Proportion	Statut
<i>beta</i>	2 525	29,47 %	Fréquente
<i>alpha</i>	2 326	27,14 %	Fréquente
<i>ductal</i>	1 077	12,57 %	Fréquente
<i>acinar</i>	958	11,18 %	Fréquente
<i>delta</i>	601	7,01 %	Fréquente
<i>activated_stellate</i>	284	3,31 %	Rare
<i>gamma</i>	255	2,98 %	Rare
<i>endothelial</i>	252	2,94 %	Rare
<i>quiescent_stellate</i>	173	2,02 %	Rare
<i>macrophage</i>	55	0,64 %	Ultra-rare
<i>mast</i>	25	0,29 %	Ultra-rare
<i>epsilon</i>	18	0,21 %	Ultra-rare
<i>schwann</i>	13	0,15 %	Ultra-rare
<i>t_cell</i>	7	0,08 %	Ultra-rare

TABLE 4 – Composition du jeu de données Baron après filtrage. Les classes rares correspondent aux types représentant moins de 5 % des cellules, et les classes ultra-rares à ceux représentant moins de 1 %.

Différentes expériences ont été menées afin de répondre à une question simple : est-ce qu'un modèle qui apprend une représentation sur certaines cellules est ensuite capable de rester performant sur des cellules qu'il n'a jamais vues ? Cette question distingue deux cadres d'évaluation. Dans un cadre transductif, toutes les cellules sont disponibles au moment de

l'apprentissage et le modèle construit ses groupes à partir de l'ensemble complet. Ce cadre est utile pour explorer un jeu de données déjà acquis, mais il ne dit pas vraiment ce qui se passerait si l'on ajoutait de nouvelles cellules ensuite. Dans un cadre inductif, au contraire, le modèle est entraîné sur une partie des cellules, puis il doit projeter ou classer de nouvelles cellules sans être réentraîné depuis le début.

Pour rendre cette évaluation possible avec des modèles basés sur des autoencodeurs, nous avons utilisé leur encodeur comme outil de projection. Cette adaptation s'appuie sur une idée déjà présente dans les méthodes de deep clustering comme DEC [34] et IDEC [35], où les cellules sont projetées dans un espace latent avant d'être rapprochées de centres de clusters. L'objectif d'appliquer un modèle entraîné à de nouvelles cellules se rapproche quant à lui davantage de la logique de projection proposée par scArches [36]. Concrètement, une fois le modèle entraîné, ses poids sont gelés : cela signifie qu'ils ne sont plus modifiés. Les cellules d'entraînement sont projetées dans l'espace latent, puis les centres des clusters appris sont calculés dans cet espace. Les nouvelles cellules passent ensuite par le même prétraitement et par le même encodeur ; elles sont finalement rattachées au cluster dont le centre est le plus proche, ou à un groupe de référence construit pendant l'entraînement. Par exemple, cela revient à apprendre les centres à partir des cellules d'entraînement, puis à attribuer les cellules du test au centre latent le plus proche. Sur Baron, nous avons conservé deux cadres principaux pour l'analyse des méthodes existantes. Le premier est transductif : l'ensemble des 8 569 cellules filtrées est analysé simultanément, puis les performances sont moyennées sur trois graines aléatoires. Le second est inductif : les cellules sont séparées en ensembles d'entraînement, de validation et de test selon un split stratifié 70/10/20. Les modèles sont ajustés sur 5 997 cellules, les hyperparamètres sont contrôlés sur 858 cellules de validation, puis les performances sont mesurées sur 1 714 cellules de test jamais vues pendant l'entraînement.

Ces deux cadres permettent de mesurer des propriétés complémentaires. Le mode transductif indique si une méthode retrouve correctement la structure globale du jeu de données complet, mais il peut être dominé par les grands types cellulaires et les batchs majoritaires. Le mode inductif teste au contraire la capacité à projeter des cellules non vues, dans un contexte où le test contient moins de cellules et où les populations rares deviennent parfois représentées par quelques exemples seulement. Dans chaque cas, les performances ont été évaluées avec des métriques globales, mais aussi avec des métriques centrées sur les petites populations. Nous avons défini les classes rares comme les types cellulaires représentant moins de 5 % des cellules du jeu Baron complet, et les classes ultra-rares comme celles représentant moins de 1 %. Ce choix de seuil est empirique : il correspond aux trois régimes visibles dans Baron, avec des classes fréquentes au-dessus de 5 %, des classes rares sous 5 % et des classes ultra-rares sous 1 %.

3.2.2 Méthodes

Nous avons alors lancé l'analyse sur un ensemble restreint de méthodes représentatives de l'état de l'art et cohérent avec notre protocole de comparaison : PCA suivie de clustering K-Means ou de Leiden, scDeepCluster, scMAE et scNAME. Les méthodes ont été exécutées avec les recommandations des auteurs lorsque celles-ci étaient disponibles, en conservant les

hyperparamètres standards afin d'éviter d'optimiser une méthode au détriment des autres. Pour PCA+Leiden, la résolution du graphe est sélectionnée automatiquement par une recherche sur grille maximisant le score de silhouette, et le nombre final de clusters n'est donc pas fixé à l'avance. PCA+KMeans, ainsi que les méthodes nécessitant un nombre de groupes, utilisent en revanche $K = 14$, correspondant au nombre de types cellulaires annotés dans Baron.

Le prétraitement reprend également celui du protocole de comparaison. Les cellules exprimant moins de 200 gènes et les gènes observés dans moins de trois cellules sont filtrés, puis les comptes sont normalisés à 20 000 par cellule et transformés par logarithme. Nous conservons ensuite les 2 000 gènes hautement variables (*highly variable genes*, HVG) selon la stratégie de Seurat, avant une mise à l'échelle des données. Ce nombre de gènes constitue un compromis standard : il réduit le bruit et le coût de calcul tout en conservant suffisamment d'information pour comparer les méthodes sur Baron.

Dans les expériences avec séparation entraînement/validation/test, la sélection des gènes variables a été réalisée à partir des données d'entraînement afin d'éviter d'utiliser indirectement de l'information provenant du test. Les expériences transductives et inductives ont été répétées trois fois avec des graines aléatoires différentes. Cette répétition reste limitée par le coût de calcul des méthodes profondes, mais elle permet déjà de réduire le risque d'interpréter un lancement isolé trop favorable ou trop défavorable.

Afin de comparer les méthodes de manière homogène, les mêmes métriques ont été calculées pour toutes les expériences. La difficulté principale vient du fait qu'un algorithme de clustering ne prédit pas directement des noms de types cellulaires mais produit seulement des groupes numérotés (par exemple cluster 0, 1 ou 2) dont les labels sont arbitraires. Sur Baron, un cluster prédit majoritairement composé de cellules *beta* doit donc être associé à l'étiquette *beta* avant de calculer une accuracy par type cellulaire.

Pour évaluer les performances de clustering, nous distinguons deux catégories de métriques, dont les formules mathématiques détaillées sont reportées en Annexe C (Tableau 10) :

Métriques de clustering global (sans alignement)

Ces métriques comparent directement la structure du clustering prédit avec la vérité terrain biologique, sans nécessiter d'attribuer un nom ou une classe aux clusters :

- **L'Indice de Rand Ajusté (ARI)** : Évalue la similarité de clustering en mesurant la proportion de paires de cellules qui sont regroupées de façon cohérente (soit ensemble, soit séparées) dans la vérité terrain et dans la prédiction, en ajustant le score pour corriger l'accord attendu par hasard. Un score de 1 indique une concordance parfaite.
- **L'Information Mutuelle Normalisée (NMI)** : Fondée sur l'entropie, elle mesure la quantité d'information partagée entre les clusters prédits et les types cellulaires réels. Normalisée entre 0 (indépendance) et 1 (clusterings identiques), elle quantifie la réduction d'incertitude sur les annotations grâce à la prédiction.

Métriques de classification (après appariement optimal)

Pour calculer des taux de réussite ou d'exactitude par type cellulaire, il faut associer chaque cluster à une étiquette biologique. Nous appliquons l'algorithme hongrois [37] pour

résoudre le problème d'affectation linéaire : il détermine l'appariement optimal entre clusters prédits et classes biologiques qui maximise le nombre total de cellules correctement attribuées. Cette approche, couramment adoptée dans des benchmarks comme scCluBench [32], permet ensuite d'évaluer :

- **L'accuracy (ACC)** : Proportion brute de cellules correctement affectées après appariement. Très sensible aux classes majoritaires, elle ne reflète pas la qualité sur les populations minoritaires.
- **La balanced accuracy (Bal ACC)** : Moyenne des rappels par classe biologique, attribuant un poids identique à chaque type cellulaire indépendamment de sa taille.
- **L'accuracy sur les classes rares (RareACC) et ultra-rares (UltraRareACC)** : Même principe que l'accuracy mais spécifique à certaines classes de cellules, les plus rares (<5%) et les ultra-rares (<1%).

Métrique de correction de batch

Lorsque le jeu de données contient plusieurs batchs exploitables, nous calculons aussi un score de correction de batch inspiré de scIB-E [38]. Ce score évalue si, dans l'espace latent ou l'embedding produit par une méthode, des cellules biologiquement similaires mais issues de batchs différents sont bien mélangées. Il agrège plusieurs critères de mélange local des batchs, comme la silhouette de batch, iLISI, kBET et la connectivité du graphe ; un score élevé indique donc que le batch influence moins la structure de l'embedding. Cette métrique doit toutefois être lue avec précaution, car une forte correction de batch peut aussi supprimer une partie du signal biologique si celui-ci est confondu avec l'origine expérimentale.

3.2.3 Résultats

La figure 2 montre d'abord que le protocole utilisé pour rendre les algorithmes inductifs fonctionne. Après entraînement sur une partie des cellules, les modèles peuvent être appliqués au jeu de test sans être réentraînés complètement, et les résultats restent proches de ceux obtenus en transductif sur le jeu complet. Dans certains cas, ils sont même meilleurs en inductif. Cette amélioration doit toutefois être interprétée prudemment : le test ne contient que 1 714 cellules, dont seulement 22 cellules ultra-rares. Quelques prédictions correctes ou incorrectes peuvent donc fortement modifier les scores sur ces populations, et une partie du gain observé peut être due à un tirage favorable.

Le second résultat important est que de bonnes métriques globales ne garantissent pas une bonne identification des cellules rares. Les scores comme le NMI, l'ARI ou l'ACC sont fortement influencés par les grands types cellulaires, comme *alpha*, *beta*, *ductal* ou *acinar*. À l'inverse, les populations rares ou ultra-rares représentent très peu de cellules ; lorsqu'elles sont manquées, leur erreur pèse donc peu dans les métriques globales. Sur Baron, plusieurs petites classes, comme *t_cell*, *epsilon*, *mast* ou *schwann*, restent ainsi mal identifiées ou confondues, ce qui apparaît surtout dans les métriques équilibrées et les taux d'erreur par type cellulaire.

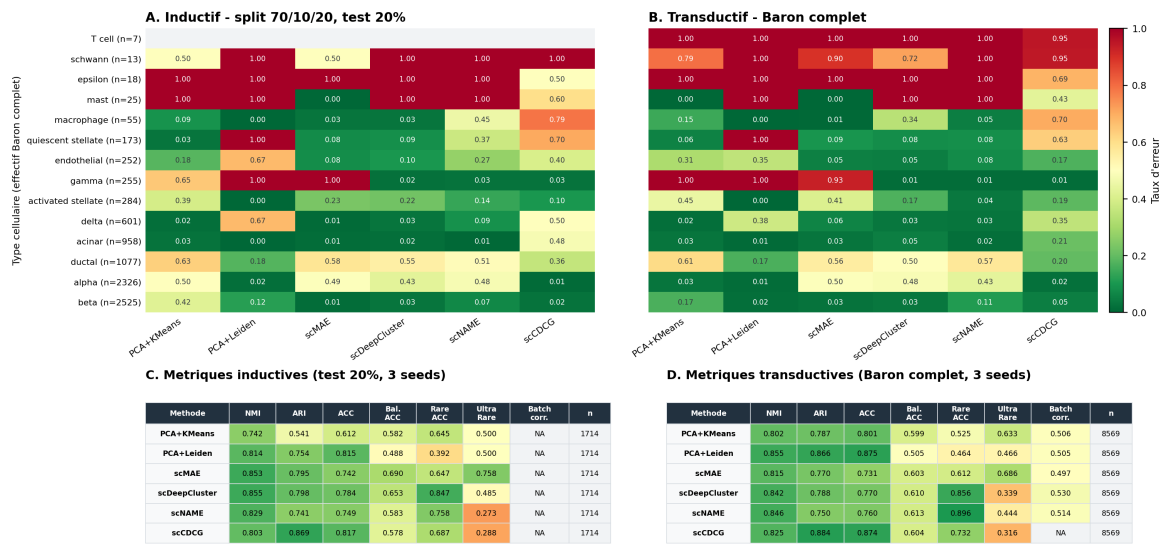
Enfin, si l'on ne regarde que les métriques globales, PCA+Leiden performe très bien, parfois mieux que les méthodes de deep learning évaluées. Ce résultat rappelle qu'une méthode simple

et bien adaptée reste un point de comparaison solide. Il ne suffit cependant pas à conclure que la problématique des cellules rares est résolue, car ces populations restent précisément celles qui contribuent le moins aux métriques classiques de la littérature pour évaluer un clustering. De plus PCA+Leiden, n'utilisant pas Kmeans, ne fixe pas un nombre attendu de cluster, on a alors que 8 clusters identifiés par leiden contre 14 attendus.

Ces observations motivent directement le développement de scRAW : l'objectif n'est pas seulement d'améliorer un score moyen, mais de construire une représentation dans laquelle les populations minoritaires conservent assez d'influence pour ne pas être noyées dans les classes dominantes.

Baron - taux d'erreur par type cellulaire et scores globaux

Erreur = 1 - rappel après appariement hongrois ; résultats moyennés sur 3 seeds. Meme pretraitement que le papier scRAW : filtrage 200/3, normalisation 20000, 2000 HVG Seurat.



Note : PCA+Leiden reprend la selection de resolution par silhouette du papier, sans forcer 14 clusters. La metrique Batch corr. est affichee lorsqu'elle est disponible ; elle n'a pas ete calculee dans le split inductif.

FIGURE 2 – Panel de résultats sur le jeu de données Baron. A–B : taux d'erreur par type cellulaire et par méthode, en mode inductif sur le test du split stratifié 70/10/20 et en mode transductif sur le jeu complet. Le taux d'erreur d'un type c vaut $1 - \text{rappel}_c$, avec $\text{rappel}_c = \text{TP}_c / (\text{TP}_c + \text{FN}_c)$ après appariement hongrois. C–D : métriques globales associées. Les deux cadres sont moyennés sur trois seeds et utilisent le prétraitement du protocole de comparaison. Pour PCA+Leiden, la résolution est sélectionnée par recherche de silhouette et le nombre de clusters n'est pas fixé à 14.

4 scRAW

4.1 Positionnement et motivation

scRAW a été développé à partir du constat présenté dans la section précédente : les méthodes de clustering peuvent obtenir de bons scores globaux tout en retrouvant mal les populations cellulaires les plus petites. L'objectif n'est donc pas seulement de construire un nouvel autoencodeur, mais de modifier la manière dont les cellules contribuent à l'apprentissage. Les cellules situées dans de petits groupes ou dans des zones peu denses de l'espace latent

reçoivent un poids plus élevé dans la loss de reconstruction pondérée $\mathcal{L}_{\text{recon,ponderee}}$. Elles influencent ainsi davantage la représentation apprise.

La méthode conserve une structure volontairement simple. Elle repose sur un autoencodeur MLP, une pondération dynamique des cellules, une contrainte locale de type triplet loss $\mathcal{L}_{\text{triplet}}$ [39] et une correction adversariale de l'information de batch $\mathcal{L}_{\text{batch}}$ lorsque plusieurs batchs sont renseignés. Dans la suite, la description correspond uniquement à la configuration par défaut de scRAW. Les autres variantes de configuration ne sont pas détaillées ici.

4.2 Pipeline

Le pipeline général de scRAW est présenté dans la figure 3. Nous appliquons aux données brutes les étapes de prétraitement standard, identiques à celles détaillées dans la comparaison de l'existant. De plus, aucune correction de batch séparée n'est appliquée à cette étape ; la prise en compte du batch intervient ensuite directement au sein du modèle.

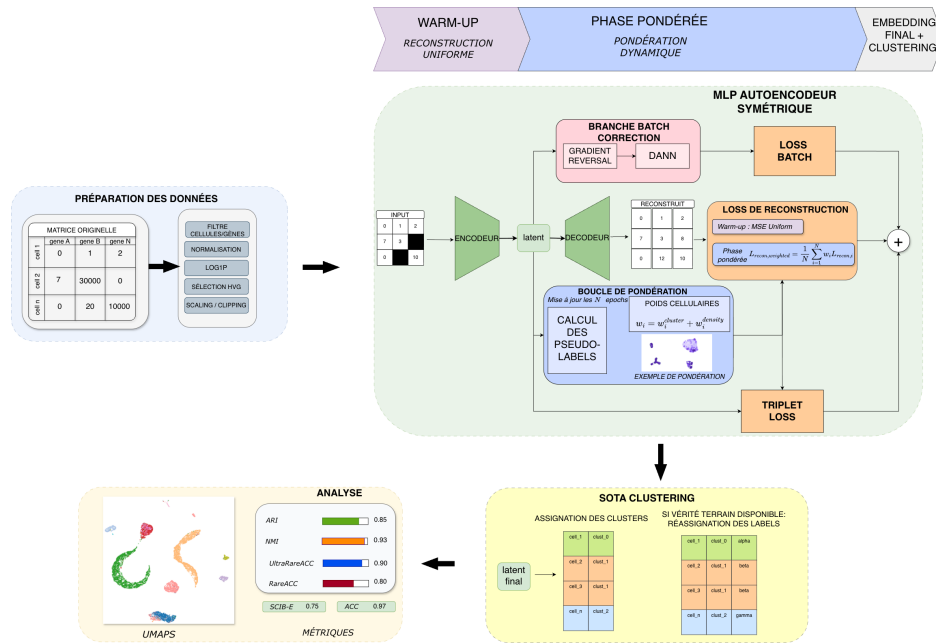


FIGURE 3 – Pipeline général de scRAW. Le modèle apprend un espace latent par autoencodeur, met à jour des poids cellulaires à partir de pseudo-clusters et de la densité locale, puis réalise le clustering final dans l'espace latent.

La matrice prétraitée est ensuite utilisée comme entrée d'un autoencodeur symétrique. Les hyperparamètres du modèle et de son entraînement ont été sélectionnés par une recherche d'hyperparamètres Optuna, dont le protocole est décrit en section 4.4, et sont détaillés en Annexe H.

L'entraînement est divisé en deux phases. Pendant les premières époques (E_{pre}), toutes les cellules contribuent de manière uniforme à la reconstruction. Cette phase de préentraînement sert à stabiliser l'espace latent avant d'introduire la pondération. À partir de l'époque E_{pre} , scRAW recalcule périodiquement (toutes les E_{upd} époques) des pseudo-labels et des poids cellulaires à partir de l'espace latent courant. Dans la configuration par défaut, les pseudo-labels sont obtenus avec Leiden. Les poids déjà présents sont lissés par une moyenne mobile

avec un momentum afin d'éviter des changements trop brusques au cours de l'apprentissage.

À la fin de l'entraînement, l'encodeur produit l'embedding final de chaque cellule. Le clustering final est ensuite réalisé avec HDBSCAN dans cet espace latent, en demandant des groupes d'au moins 8 cellules et une estimation locale fondée sur 6 voisins. Les points considérés comme bruit par HDBSCAN ne sont pas réassignés dans cette configuration. Les sorties principales de scRAW sont donc l'espace latent, les labels de clusters prédits, les poids cellulaires finaux, le modèle entraîné, l'historique des termes de loss $\mathcal{L}$ et les métriques calculées par le pipeline d'évaluation.

4.3 Fonctions de loss $\mathcal{L}$ et variantes

Dans la configuration par défaut, l'objectif optimisé combine trois termes de loss $\mathcal{L}$:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon,ponderee}} + \rho(e)\lambda_{\text{triplet}}\mathcal{L}_{\text{triplet}} + \lambda_{\text{batch}}\mathcal{L}_{\text{batch}}. \quad (1)$$

Le terme $\mathcal{L}_{\text{recon,ponderee}}$ reconstruit les profils d'expression, $\mathcal{L}_{\text{triplet}}$ structure localement l'espace latent autour des cellules rares ou isolées, et $\mathcal{L}_{\text{batch}}$ réduit l'information de batch visible dans la représentation lorsque plusieurs batchs sont présents. Le facteur $\rho(e)$ augmente progressivement le poids effectif de $\mathcal{L}_{\text{triplet}}$ au début de son activation.

Reconstruction pondérée $\mathcal{L}_{\text{recon,ponderee}}$

La loss de reconstruction $\mathcal{L}_{\text{recon}}$ est le terme principal de scRAW. Dans cette configuration, il s'agit d'une erreur quadratique moyenne (MSE) calculée entre la matrice prétraitée X et sa reconstruction $\hat{X}$. Pour une cellule i , on peut l'écrire simplement :

$$\mathcal{L}_{\text{recon},i} = \frac{1}{G} \sum_{g=1}^G (\hat{X}_{i,g} - X_{i,g})^2, \quad (2)$$

où G est le nombre de gènes conservés après prétraitement. Une fraction prédéfinie des entrées est masquée aléatoirement pendant l'entraînement, avec une valeur de masquage nulle. Les positions masquées reçoivent un poids interne spécifique dans le calcul de $\mathcal{L}_{\text{recon},i}$. Ce masquage est aussi actif pendant la phase pondérée.

La différence principale avec un autoencodeur standard vient du facteur ω_i , qui module la contribution de chaque cellule :

$$\mathcal{L}_{\text{recon,ponderee}} = \frac{1}{N} \sum_{i=1}^N \omega_i \mathcal{L}_{\text{recon},i}. \quad (3)$$

Pendant la phase initiale, $\omega_i = 1$ pour toutes les cellules. Pendant la phase pondérée, ω_i est recalculé à partir de deux informations. La première est la taille du pseudo-cluster Leiden auquel appartient la cellule : plus le pseudo-cluster est petit, plus le poids augmente. La seconde est la densité locale dans l'espace latent, estimée par la distance au voisin d'ordre k : une cellule plus isolée reçoit un poids plus élevé. Dans la configuration par défaut, ces deux

composantes sont fusionnées de manière multiplicative :

$$\tilde{\omega}_i = \omega_i^{\text{cluster}} \times \omega_i^{\text{densite}}. \quad (4)$$

Le résultat est normalisé pour avoir une moyenne proche de 1, puis limité entre des bornes de sécurité. Ces bornes évitent qu’une cellule domine entièrement $\mathcal{L}_{\text{recon,ponderee}}$, tout en empêchant les cellules très fréquentes de disparaître de l’apprentissage.

Triplet loss rare-aware $\mathcal{L}_{\text{triplet}}$

La *triplet loss* $\mathcal{L}_{\text{triplet}}$ est une loss de comparaison relative dans l’espace latent. Elle ne cherche pas à reconstruire directement les profils d’expression ; elle agit plutôt sur l’organisation géométrique des embeddings. Pour un triplet composé d’une cellule ancre a , d’une cellule positive p supposée similaire, et d’une cellule négative n supposée différente, elle pénalise le modèle lorsque l’ancre n’est pas suffisamment plus proche de p que de n .

Ce type de loss $\mathcal{L}_{\text{triplet}}$ est utilisé pour apprendre des représentations où la proximité a un sens, et il a aussi été adapté au clustering scRNA-seq ; par exemple, scHNTL utilise une triplet loss avec des voisins d’ordre élevé pour rapprocher les cellules considérées voisines et éloigner les cellules non voisines [40].

Dans scRAW, ce principe est appliqué uniquement aux cellules rares. Le terme $\mathcal{L}_{\text{triplet}}$ est activé à partir d’une époque prédéfinie (E_{triplet}), avec un poids λ_{triplet} , une marge déterminée et une montée progressive contrôlée par $\rho(e)$. Les ancres sont les cellules dont le poids est supérieur ou égal à un seuil d’ancrage, dans la limite d’un nombre maximal d’ancres par mini-batch. Pour chaque ancre, la méthode cherche une cellule positive dans le même pseudo-cluster et une cellule négative dans un autre pseudo-cluster. Cette loss rend donc les groupes rares plus visibles localement dans l’espace latent : elle les rapproche de cellules similaires sans dépendre uniquement du signal global de reconstruction, et les sépare mieux des grands groupes voisins.

Correction adversariale de batch $\mathcal{L}_{\text{batch}}$

Dans scRAW, la correction de batch suit le principe des DANN (*Domain-Adversarial Neural Networks*) [30]. Le modèle peut être vu comme deux objectifs branchés sur le même espace latent $z_i = E(x_i)$. L’autoencodeur utilise cet espace pour reconstruire les profils d’expression et organiser les cellules, tandis qu’un petit classifieur de batch reçoit le même vecteur latent et essaie de prédire l’origine expérimentale b_i de la cellule.

Le terme $\mathcal{L}_{\text{batch}}$ correspond donc à la pénalisation associée à cette prédiction de batch. Le classifieur est entraîné normalement : il minimise $\mathcal{L}_{\text{batch}}$ pour reconnaître au mieux les batches. En revanche, entre l’espace latent et ce classifieur, une couche d’inversion de gradient laisse passer les valeurs sans changement lors de la propagation avant, mais inverse le signe du gradient lors de la rétropropagation. Ainsi, le classifieur apprend à détecter les batches, tandis que l’encodeur reçoit le signal opposé et apprend à produire une représentation dans laquelle cette origine devient moins reconnaissable. L’objectif n’est donc pas d’appliquer une correction externe aux données d’entrée, mais d’apprendre un espace latent qui conserve l’information

utile au clustering tout en réduisant les séparations dues au protocole, au donneur ou à la technologie.

Dans la configuration par défaut, cette branche est utilisée lorsque la colonne de batch contient au moins deux batches après filtrage, avec un poids λ_{batch} . La loss $\mathcal{L}_{\text{batch}}$ est activée à une époque déterminée (E_{batch}) et augmente progressivement sur un nombre d'époques déterminé, afin de laisser d'abord l'autoencodeur apprendre une représentation stable.

L'architecture du modèle présenté ici a été sélectionnée par la recherche d'hyperparamètres décrite dans la section suivante. Une étude d'ablations, dont les résultats sont disponibles en Annexe G, évalue ensuite l'importance des principaux éléments de l'algorithme.

4.4 Recherche d'hyperparamètres

La configuration par défaut de scRAW a été construite progressivement. L'objectif n'était pas de trouver le meilleur réglage sur un seul jeu de données, mais d'identifier une configuration assez stable pour être utilisée sans réajustement fin sur un nouveau jeu de données. Les paramètres explorés concernaient principalement l'architecture de l'autoencodeur, le rythme d'entraînement, la pondération des cellules dans $\mathcal{L}_{\text{recon,ponderee}}$, la triplet loss $\mathcal{L}_{\text{triplet}}$, la correction adversariale de batch $\mathcal{L}_{\text{batch}}$ et le clustering final.

Les recherches ont été conduites avec Optuna [41]. Chaque essai correspond à une configuration complète : Optuna propose un ensemble d'hyperparamètres, scRAW est entraîné avec cette configuration, puis les métriques de clustering sont calculées. Le score utilisé sert uniquement à guider la recherche. Il combine des métriques globales et des métriques sensibles aux classes rares, afin d'éviter de sélectionner une configuration qui améliore seulement les grands groupes cellulaires.

La première phase a été une recherche large sur Baron Human Pancreas. Elle a servi à explorer un espace de paramètres volontairement large et à identifier une première famille de configurations pertinentes. Cette étape a donné un point de départ, mais elle ne suffisait pas à définir une configuration finale, car elle restait centrée sur un seul jeu de données.

La deuxième phase a donc évalué les configurations sur plusieurs jeux de données. L'espace de recherche a été réduit à partir des meilleurs comportements observés sur Baron, puis testé sur huit jeux de données d'entraînement. Ces jeux de données, présentés en détail dans l'Annexe B, ont été sélectionnés pour représenter une grande diversité biologique et technique, avec divers degrés de déséquilibre de classe (indice de Gini) et d'effets de batch (PCR sur PCA). Le score agrégé tenait compte à la fois de la performance moyenne et des jeux les moins bien résolus, afin de favoriser les réglages robustes plutôt que les réglages très bons sur quelques cas seulement.

La dernière phase a encore resserré l'espace de recherche autour des configurations les plus stables. Elle a conservé les choix méthodologiques finalement retenus pour la configuration par défaut : reconstruction MSE $\mathcal{L}_{\text{recon}}$, transformation logarithmique, pondération dynamique $\mathcal{L}_{\text{recon,ponderee}}$, triplet loss $\mathcal{L}_{\text{triplet}}$, correction adversariale de batch $\mathcal{L}_{\text{batch}}$ et clustering final hybride autour de HDBSCAN. Cette étape a permis de fixer la configuration utilisée dans les expériences du rapport. Les valeurs exactes des hyperparamètres retenus sont données en Annexe H, et les figures d'importance des paramètres sont regroupées en Annexe I.

4.5 Résultats et discussion

L'évaluation principale de scRAW a d'abord été conduite sur le même ensemble de huit jeux de données du sous-ensemble commun (détaillé en Annexe B). L'objectif était de vérifier si scRAW, avec une configuration stable sur plusieurs jeux de données, pouvait se comparer aux méthodes de l'état de l'art. La comparaison inclut des méthodes orientées vers les populations rares, des méthodes généralistes et des méthodes ou pipelines de correction de batch. Comme ces jeux de données ont également servi à stabiliser les hyperparamètres, cette analyse est complétée ensuite par deux jeux de validation externe non utilisés pendant l'optimisation.

Pour faciliter la lecture, la figure 4 affiche scRAW ainsi que les meilleures méthodes observées pour chaque métrique. Les résultats complets, incluant les méthodes non affichées et les variantes avec correction de batch externe, sont fournis en Annexe D, avec une synthèse des familles utilisées dans le Tableau 11. Ces variantes sont conservées comme analyses complémentaires, car elles ne correspondent pas toujours au protocole proposé dans les articles originaux des méthodes comparées.

La figure 4 montre surtout que les méthodes les plus performantes sur une métrique ne le sont pas nécessairement sur les autres. Certaines méthodes dépassent scRAW sur un critère précis, mais elles présentent généralement une baisse plus marquée sur un autre aspect de l'évaluation.

Par exemple, scaIDE obtient de très bons scores globaux en ARI ou en ACC, mais ses scores sur les populations ultra-rares restent plus faibles que ceux de scRAW. À l'inverse, scCAD atteint une UltraRareACC légèrement supérieure, mais ses scores globaux en ARI et en ACC sont nettement plus bas. Le même type de compromis apparaît pour la correction de batch : scVI et Harmony corrigent mieux l'effet de batch, mais scVI retrouve moins bien les populations ultra-rares, tandis que Harmony reste plus faible sur les métriques centrées sur les classes rares.

La matrice de rangs de la Figure 5 formalise ce compromis. Les rangs sont calculés à partir des moyennes obtenues par méthode sur les huit jeux de données communs, avec le rang 1 attribué au meilleur score de chaque métrique. scRAW n'est pas premier sur une métrique isolée, mais reste dans le top 3 sur les cinq critères, ce qui traduit une stabilité transversale. À l'inverse, scaIDE est premier en ARI et en ACC mais chute sur les métriques rares, tandis que scCAD est premier en UltraRareACC mais très bas sur les métriques globales.

scRAW n'est donc pas systématiquement la meilleure méthode pour chaque métrique prise séparément. En revanche, ses scores restent élevés sur l'ensemble des critères évalués : 0,658 en ARI, 0,741 en ACC, 0,636 en RareACC, 0,641 en UltraRareACC et 0,636 pour la correction de batch calculée uniquement sur les jeux concernés par un effet de batch. Cette stabilité en fait le meilleur compromis observé dans ces expériences pour conserver les populations rares, limiter l'effet de batch et maintenir un clustering global précis. Les analyses complémentaires montrent aussi que l'ajout d'une correction de batch peut améliorer certaines méthodes, soit lorsqu'elle est appliquée en prétraitement, soit lorsqu'elle intervient après l'apprentissage dans l'espace latent.

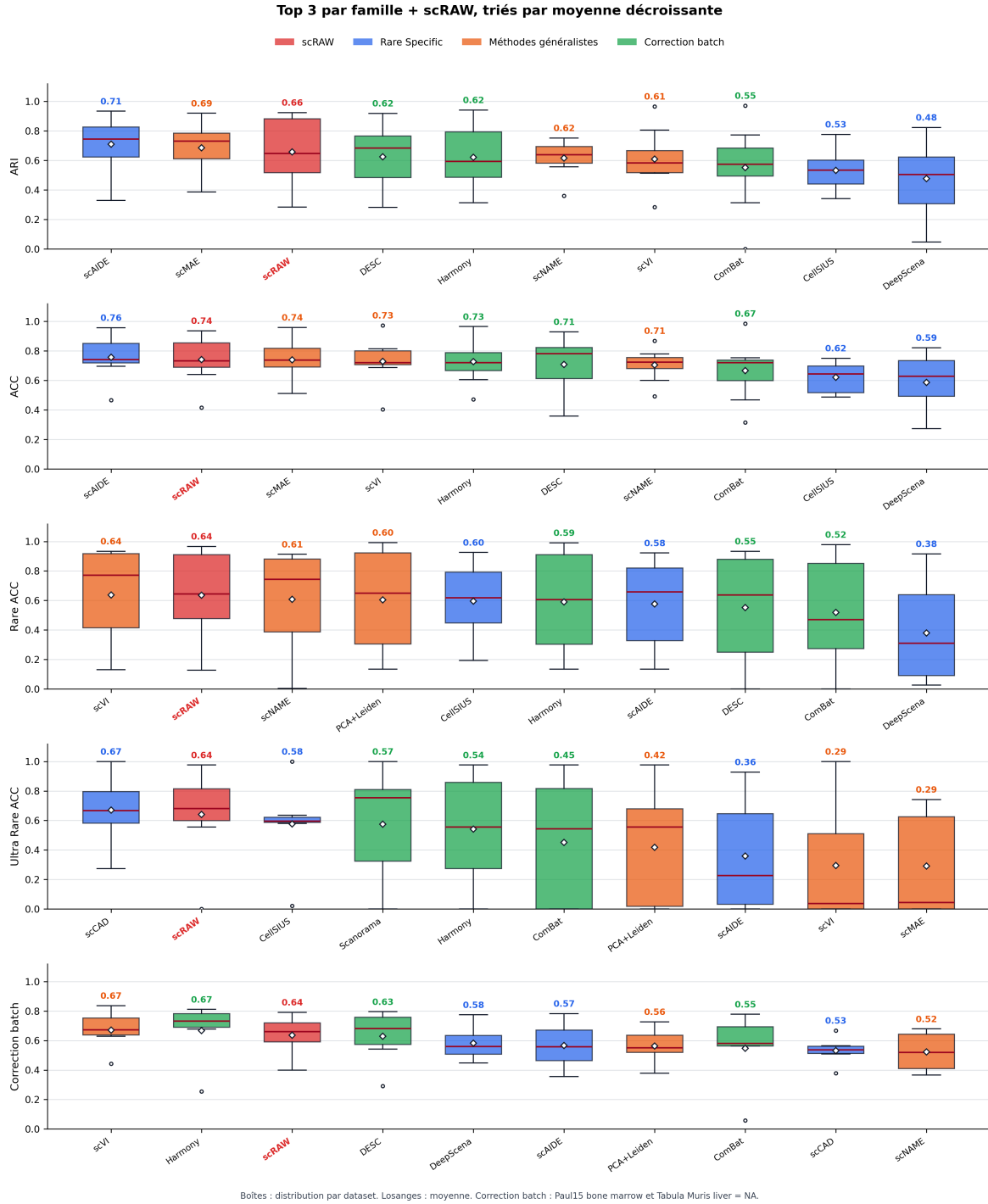


FIGURE 4 – Distribution des performances sur les huit jeux de données communs pour scRAW et le top 3 de chaque famille de méthodes. Chaque panneau correspond à une métrique (de haut en bas : ARI, ACC, Rare ACC, Ultra Rare ACC, Correction batch). Les couleurs identifient les familles de méthodes : scRAW (rouge), **Rare Specific** (bleu), **Méthodes généralistes** (orange) et **Correction batch** (vert). Pour chaque métrique, les méthodes représentées (scRAW et les trois meilleures de chaque famille) sont triées globalement par moyenne décroissante (meilleure à gauche). Les boîtes représentent la distribution des scores par jeu de données, les points noirs les valeurs individuelles et le losange blanc la moyenne ; pour l'Ultra Rare ACC, les jeux sans cellules ultra-rares ne contribuent pas au calcul, et pour la correction de batch, les jeux sans effet de batch exploitable sont indiqués comme NA et exclus du calcul.

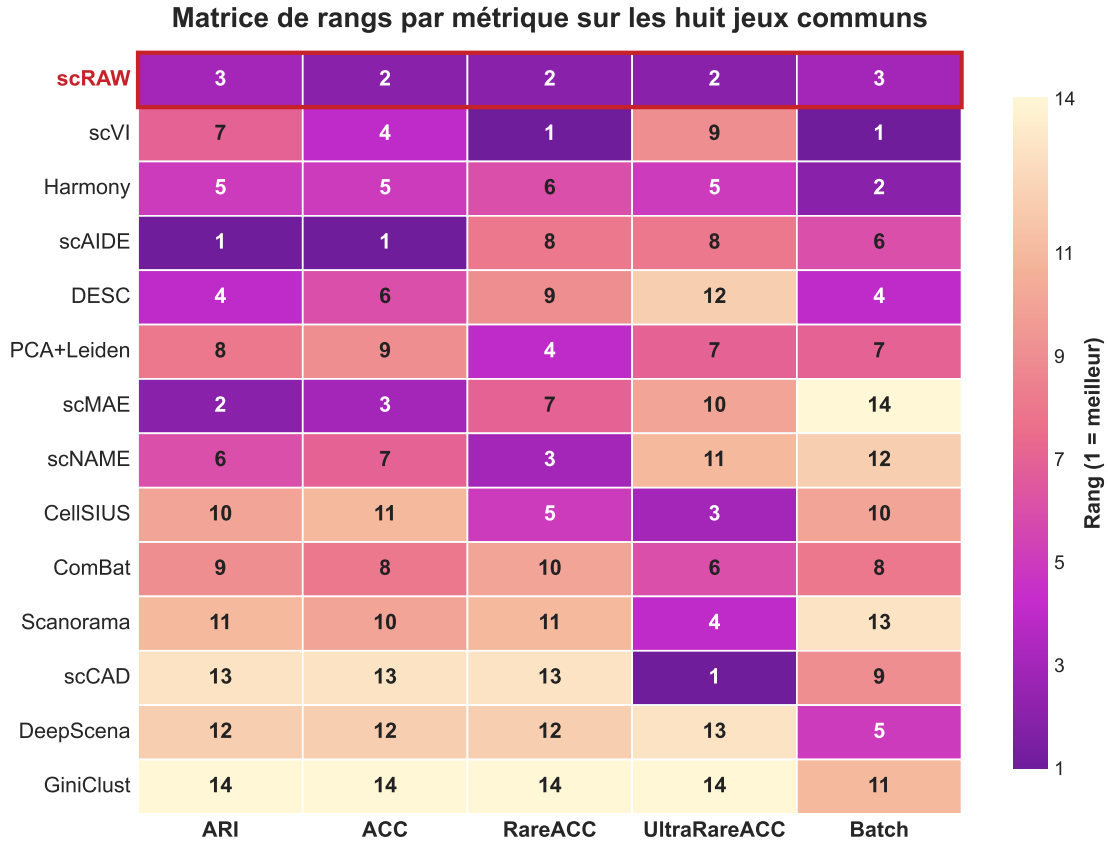


FIGURE 5 – **Matrice de rangs des algorithmes selon les métriques moyennes sur le jeu commun common-8.** Les lignes correspondent aux méthodes et les colonnes aux métriques affichées dans la section : ARI, ACC, RareACC, UltraRareACC et correction de batch. Le rang 1 correspond à la meilleure moyenne pour une métrique donnée. Les cases suivent un dégradé du violet (meilleur rang) au jaune clair (rang moins favorable). La matrice ne contient pas de colonne de score moyen ni de rang global, afin de faire apparaître directement les compromis métrique par métrique.

4.5.1 Validation sur des jeux non vus pendant l'optimisation

Afin de compléter cette évaluation, la configuration finale de scRAW a aussi été testée sur deux jeux de données qui n'ont pas été utilisés pendant la recherche d'hyperparamètres : **Human Pancreas**, issu d'OpenProblems, et **SCIB Mouse Pancreas 1**. Ces deux jeux conservent le même protocole de prétraitement et les mêmes métriques que l'évaluation principale. Human Pancreas présente un effet de batch technique marqué entre protocoles de séquençage, tandis que SCIB Mouse Pancreas 1 est un petit jeu de données sans batch explicite et avec plusieurs classes très faiblement représentées.

Le tableau 5 présente les résultats obtenus par scRAW avec la configuration par défaut, sans nouvelle recherche d'hyperparamètres spécifique à ces jeux de données, ainsi que par les méthodes de comparaison retenues. Pour chaque jeu de données, les méthodes sont triées par moyenne décroissante sur les cinq métriques affichées, afin de conserver une lecture synthétique des compromis entre performance globale, populations rares et correction de batch.

Sur Human Pancreas, scRAW reste proche des meilleures méthodes sur les métriques globales et rares, sans dominer la correction de batch. Les meilleurs scores isolés sont obtenus

Dataset	Méthode	ARI	ACC	RareACC	UltraRareACC	Batch
Human Pancreas	Harmony	<u>0,817</u>	0,897	0,848	0,662	0,669
	ComBat	0,839	0,892	0,832	0,628	0,587
	scRAW	0,791	<u>0,817</u>	0,829	0,622	0,511
	scVI	0,595	0,720	0,811	0,581	0,530
	Scanorama	0,432	0,563	0,854	0,757	0,610
	PCA+Leiden	0,444	0,513	0,878	0,730	0,478
	DeepScena	0,626	0,713	0,716	0,392	0,462
	CellSIUS	0,444	0,519	0,665	<u>0,750</u>	0,485
	scAIDE	0,520	0,564	0,808	<u>0,574</u>	0,377
	scMAE	0,502	0,543	0,817	0,595	0,353
	DESC	0,474	0,511	0,784	0,622	0,304
	scCAD	0,346	0,311	0,466	0,716	0,484
	scNAME	0,502	0,542	0,646	0,216	0,224
	GiniClust	0,042	0,342	0,021	0,047	0,485
SCIB Mouse Pancreas 1	scMAE	0,560	0,614	0,686	0,706	NA
	scRAW	0,821	<u>0,787</u>	0,721	0,235	NA
	scCAD	0,636	0,634	0,628	0,471	NA
	CellSIUS	0,588	0,674	0,756	<u>0,294</u>	NA
	DESC	0,781	0,839	0,558	0,000	NA
	scAIDE	<u>0,672</u>	0,715	0,523	0,471	NA
	Harmony	0,463	0,551	0,756	<u>0,235</u>	NA
	Scanorama	0,463	0,551	0,756	0,235	NA
	PCA+Leiden	0,463	0,551	0,756	0,235	NA
	ComBat	0,462	0,550	0,756	0,235	NA
	scVI	0,491	0,578	0,686	0,000	NA
	scNAME	0,431	0,468	0,744	0,294	NA
	DeepScena	0,233	0,393	<u>0,128</u>	0,176	NA
	GiniClust	0,021	0,443	0,058	0,000	NA

TABLE 5 – Résultats sur deux jeux de données non vus pendant l’optimisation des hyperparamètres. Les méthodes sont triées, pour chaque jeu de données, par moyenne décroissante des cinq métriques affichées. Les meilleures valeurs par métrique et par jeu de données sont indiquées en gras et les deuxièmes meilleures sont soulignées ; scRAW est affiché en rouge. Pour SCIB Mouse Pancreas 1, le score de correction de batch est indiqué comme NA car le jeu ne contient qu’un batch explicite.

par des méthodes différentes selon la métrique : ComBat pour l’ARI, Harmony pour l’ACC et la correction de batch, PCA+Leiden pour la RareACC, et Scanorama pour l’UltraRareACC. Sur SCIB Mouse Pancreas 1, scRAW obtient la meilleure ARI et une ACC élevée, mais l’UltraRareACC reste limitée. Ce résultat doit être lu avec prudence, car les populations ultra-rares y contiennent seulement quelques cellules. Ces deux validations externes fournissent donc un premier contrôle sur des jeux non vus pendant l’optimisation : scRAW y conserve un comportement compétitif, mais la meilleure méthode dépend encore fortement de la métrique et du jeu de données.

Enfin, scRAW reste plus coûteux que les pipelines classiques, avec un temps d’exécution moyen d’environ 794 secondes contre quelques secondes pour PCA+Leiden ou Harmony. Ces résultats soutiennent l’hypothèse principale de scRAW : augmenter l’influence des cellules rares pendant l’apprentissage permet de mieux préserver les populations minoritaires sans sacrifier fortement la performance globale. La méthode reste cependant à consolider par davantage de répétitions, de nouveaux jeux de données, une analyse plus fine des erreurs par type cellulaire et une validation biologique complète.

5 Travaux complémentaires

5.1 Transfert de loss

Cette expérience vise à tester si la pondération de la loss de reconstruction $\mathcal{L}_{\text{recon,ponderee}}$ utilisée dans scRAW peut être intégrée à d'autres algorithmes de deep learning disposant d'une loss de reconstruction $\mathcal{L}_{\text{recon}}$ compatible. L'objectif est de distinguer l'effet de la pondération rare-aware de l'architecture complète de scRAW. Les méthodes testées sont scMAE, scDeepCluster et DESC.

Pour chaque méthode, la baseline correspond à l'algorithme de base avec ses hyperparamètres auteur. Les variantes pondérées conservent cette base, mais ajoutent une pondération dans la loss de reconstruction $\mathcal{L}_{\text{recon}}$, ce qui donne une version $\mathcal{L}_{\text{recon,ponderee}}$. Comme dans la configuration par défaut de scRAW, cette pondération n'est pas activée dès le début de l'entraînement : une phase de warm-up est conservée, puis la pondération démarre à l'époque 55. Pour les variantes avec triplet loss $\mathcal{L}_{\text{triplet}}$, cette composante est activée à partir de l'époque 60.

Plusieurs configurations ont été testées : pondération par densité seule, pondération complète avec pseudo-labels Leiden ou KMeans, et variantes ajoutant $\mathcal{L}_{\text{triplet}}$. La figure 6 ne présente donc pas toutes les configurations, mais seulement la baseline et la meilleure variante pondérée pour chaque algorithme. Cette meilleure variante est choisie selon la moyenne de trois métriques : ARI, RareACC et UltraRareACC. Pour avoir des détails sur les configurations testées, la configuration retenue choisie et la table des résultats correspondante, voir l'Annexe E.

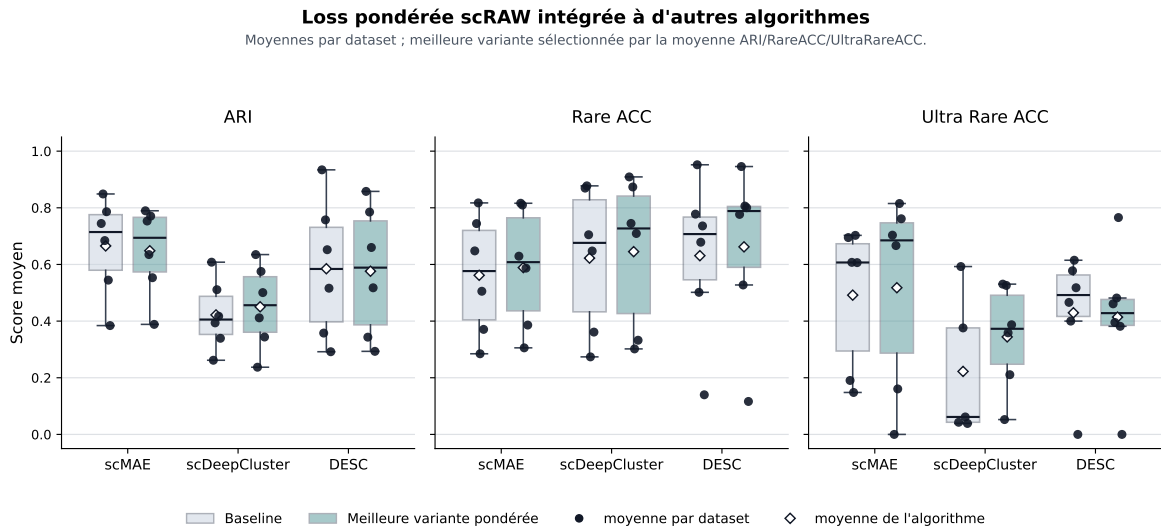


FIGURE 6 – Comparaison entre les algorithmes de base et leur meilleure variante avec loss pondérée scRAW. Les scores sont des moyennes par jeu de données.

Les résultats montrent un effet dépendant de l'algorithme. Pour scDeepCluster, la variante pondérée améliore les trois métriques retenues, avec un gain marqué sur UltraRareACC. Pour scMAE, la pondération améliore RareACC et UltraRareACC, mais l'ARI diminue légèrement. Pour DESC, la meilleure variante améliore RareACC, tandis que l'ARI et UltraRareACC

restent légèrement inférieurs à la baseline. Ces résultats indiquent que la pondération peut être transférée à d'autres modèles, mais que son effet dépend de l'architecture et du mode de clustering final.

5.2 Induction

Le passage à un cadre inductif permet d'évaluer si une représentation apprise sur une partie des cellules peut être appliquée à un batch non vu, sans réentraîner entièrement le modèle. Cette situation se rapproche d'un usage réel : entraîner un modèle sur des patients déjà caractérisés, le conserver, puis l'appliquer directement à un nouveau patient. Dans Baron Human Pancreas, par exemple, le modèle est entraîné sur une partie des donneurs puis évalué sur le donneur laissé de côté. Cette logique est utile lorsque réentraîner un modèle à chaque nouveau patient serait coûteux et moins reproductible.

Ici, le protocole général suit une séparation par groupe biologique ou technique : patient, donneur, individu macaque, batch ou technologie selon le jeu de données. Pour chaque split, les méthodes sont entraînées uniquement sur les groupes d'entraînement, puis les métriques sont calculées sur le groupe test non vu. La figure 7 compare scRAW en mode inductif aux trois méthodes profondes retenues dans cette expérience : scNAME, scMAE et scDeepCluster.

Les résultats sont agrégés par jeu de données : les splits disponibles sont d'abord moyennés, puis les boxplots sont construits à partir de ces moyennes. Chaque point noir correspond donc à un jeu de données, la boîte représente la distribution des moyennes disponibles pour un algorithme, et le losange blanc indique sa moyenne globale. Les résultats détaillés par jeu de données et la liste des splits sont fournis en Annexe F.

Sur ces moyennes par jeu de données, scRAW obtient une moyenne de 0,787 en ARI, 0,826 en ACC, 0,752 en RareACC et 0,790 en UltraRareACC. La différence par rapport aux autres méthodes est surtout marquée sur l'UltraRareACC, où ces dernières restent plus instables et nettement plus basses en moyenne. La RareACC est plus contrastée : certains jeux de données restent difficiles, notamment Kang PBMC, mais scRAW conserve une distribution globalement élevée. Ces résultats suggèrent que notre algorithme reste efficace dans un cadre inductif et qu'il garde sa fiabilité pour l'identification de populations rares et ultra-rares sur des jeux de données déséquilibrés.

5.3 Interprétation biologique

Une analyse biologique a été réalisée sur Baron Human Pancreas avec scRAW. L'objectif est de vérifier la cohérence des clusters prédits par scRAW avec les types cellulaires de référence en s'appuyant sur les gènes marqueurs, comme résumé dans la figure 8. Inspiré des protocoles de caractérisation biologique de scCluBench [32] et de scAIDE [24], ce protocole de validation extrait et compare les signatures d'expression des clusters prédits avec les types cellulaires de référence. Pour chaque type cellulaire de la vérité terrain et pour chaque cluster prédit, les 100 meilleurs gènes différentiellement exprimés (DEG) sont extraits à l'aide d'un test de Wilcoxon bilatéral. Ce calcul statistique est effectué après avoir filtré les gènes exprimés dans moins de 3 cellules, normalisé la somme des comptes par cellule à une cible de 10 000 et appliqué une transformation logarithmique ($\log(x + 1)$). En pratique, le package Scanpy

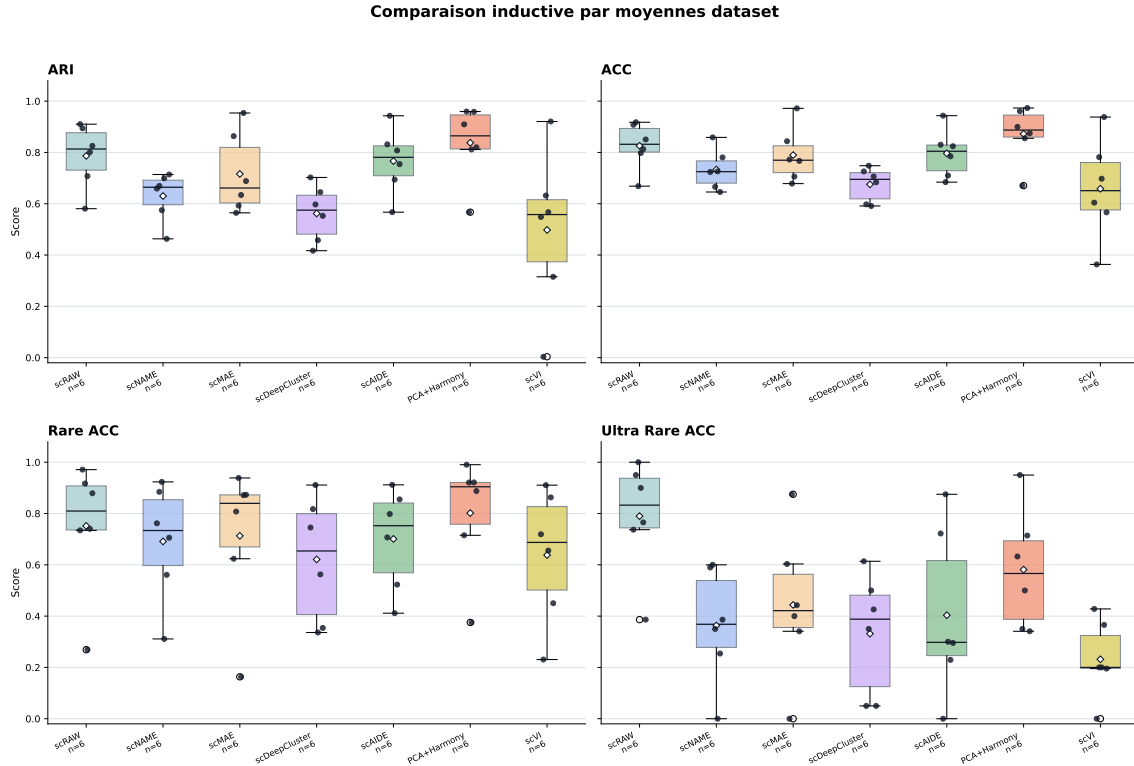


FIGURE 7 – Comparaison inductive de scRAW avec scNAME, scMAE et scDeepCluster sur six jeux de données. Chaque boîte représente la distribution des moyennes par jeu de données, et chaque point correspond à un jeu de données. Les losanges indiquent la moyenne globale de l’algorithme pour la métrique considérée.

calcule les statistiques de test et les p-values pour tous les gènes, les trie par significativité (statistique de test ou p-value la plus faible), puis renvoie les 100 premiers gènes sans appliquer de filtre de p-value. Les clusters sont ensuite annotés en les affectant au type cellulaire qui maximise le recouvrement (l’intersection) de leurs gènes marqueurs respectifs, en complément de l’assignation hongroise classique.

Sur Baron, cette annotation par recouvrement atteint une accuracy de 94,8% et une balanced accuracy de 88,9%, contre 93,4% et 87,0% pour le mapping hongrois. Les deux annotations concordent pour 97,4% des cellules, ce qui indique que le clustering réalisé par scRAW est globalement en accord avec les marqueurs de référence. La projection t-SNE montre cependant que certains types cellulaires peuvent être fragmentés en deux sous-clusters (figure 8a), tandis que la heatmap de recouvrement précise la correspondance entre clusters prédits et types annotés (figure 8b). Il convient de noter que cette heatmap indique la proportion de gènes marqueurs partagés et non un mélange de cellules : par exemple, le cluster 5 (lymphocytes T) partage 33% de ses marqueurs avec les mastocytes en raison de leur signature immunitaire commune, bien que l’algorithme sépare parfaitement leurs cellules, comme le montre le panneau t-SNE de la figure 8a.



5.4 Limites méthodologiques

Le protocole conserve également quelques dépendances à l’annotation de référence. Certaines méthodes nécessitent un nombre de groupes attendu ou une résolution de clustering choisie en fonction du nombre de types cellulaires connus. Enfin, l’analyse biologique sur Baron montre une cohérence entre clusters et marqueurs, mais elle reste préliminaire : elle ne suffit pas encore à démontrer que chaque population rare détectée correspond à un état cellulaire robuste et interprétable.

Conclusion

La contribution principale du stage est **scRAW**, une méthode fondée sur un autoencodeur et une pondération dynamique des cellules rares ou isolées. Les expériences menées sur plusieurs jeux de données montrent que cette stratégie permet d’obtenir un compromis intéressant entre

performance globale, identification des populations rares et prise en compte de l'effet de batch. Les résultats ajoutés sur deux jeux non utilisés pendant l'optimisation des hyperparamètres apportent un premier signal de robustesse externe. Les analyses complémentaires suggèrent aussi que la logique de pondération peut être transférée à d'autres modèles et que scRAW conserve un comportement prometteur dans un cadre inductif.

Les perspectives principales sont donc de consolider ces résultats sur davantage de répétitions, de splits et de jeux de données indépendants, en couvrant plus de tissus, d'espèces et de niveaux de déséquilibre. Il faudrait aussi rendre le protocole plus strictement non supervisé, en sélectionnant le nombre de clusters à partir de critères internes, puis approfondir la validation biologique par expression différentielle, enrichissement fonctionnel et comparaison à des bases de marqueurs. Ces étapes permettraient de mieux distinguer les vrais sous-types cellulaires des fragments de clusters dus au bruit, à l'effet de batch ou aux choix de clustering.

Glossaire et abréviations

ACC Accuracy, proportion de cellules correctement attribuées après appariement des clusters.

ARI Adjusted Rand Index, métrique de similarité entre deux clusterings.

Batch Ensemble de cellules partageant une même origine technique ou expérimentale.

Contrastive learning Apprentissage par contraste, méthode d'apprentissage visant à rapprocher les représentations de cellules similaires et à éloigner celles de cellules différentes.

HVG Highly Variable Genes, gènes hautement variables conservés après prétraitement.

Masked autoencodeur Autoencodeur masqué, architecture entraînée à prédire des parties manquantes des données d'entrée pour apprendre des représentations robustes.

NMI Normalized Mutual Information, métrique d'information partagée entre clusters et annotations.

PCA Principal Component Analysis, réduction linéaire de dimension.

scRNA-seq Single-cell RNA sequencing, séquençage de l'ARN à l'échelle de la cellule unique.

UMAP Uniform Manifold Approximation and Projection, méthode de projection utilisée pour visualiser les cellules.

VAE Variational Autoencoder, modèle d'apprentissage profond probabiliste qui projette les données sous forme de distributions de probabilité dans l'espace latent.

A Structure et organigramme du LaBRI

Le Laboratoire Bordelais de Recherche en Informatique (LaBRI), unité mixte de recherche UMR 5800 associée au CNRS, à l'Université de Bordeaux et à Bordeaux INP, en partenariat avec Inria, est structuré autour de plusieurs grands pôles :

- **Direction** : Pascal Desbarats (Directeur par intérim), Nicolas Hanusse (Directeur adjoint par intérim), Serge Chaumette (Chargé de mission transfert et valorisation).
- **Administration** (Administratrice d'unité : Christine Richard) :
 - *Équipe administrative* (resp. Cathy Roubineau) : Accueil (Claire Douvier, Laurence Pinosa), Assistante administrative (Safia Amsili), Assistante de communication (Auriane Dantès).
 - *Équipe financière* (resp. Chloé Godard) : Gestionnaires (Isabelle Garcia, Anthony Gau, Emmanuelle Lesage, Elia Meyre).
 - *Assistante du SCRIME* : Annick Mersier.
- **Recherche** (structurée en plusieurs départements) :
 - *Image et Son* (Fabien Baldacci)
 - *Combinatoire et Algorithmique* (Cyril Gavaille)
 - *Systèmes et Données* (Jean-Rémy Falleri), département auquel appartient l'équipe BKB.
 - *Méthodes et Modèles formels* (Laurent Bienvenu)
 - *Supports et AlgoriThmes pour les Applications Numériques hAutes performanceS* (Pierre Ramet)
- **Appui à la recherche** :
 - *Ingénieurs affectés sur projet* : Nathalie Furmento, Frédéric Lalanne, Boris Mansencal, Gérald Point.
 - *Équipe soutien "Systèmes - Réseaux"* (resp. Pascal Ung) : Ingénieurs ASR (Michaël Fontaine, Emmanuel Fontaine, Gabriel Larrouy), Technicien ASR (Alexandre Issouli), Ingénieur DEV-Web (Pierre Lacroix).
- **Autre** : Assistant de prévention (Gérald Point).

L'organigramme structurel simplifié correspondant est présenté dans la Figure 9.

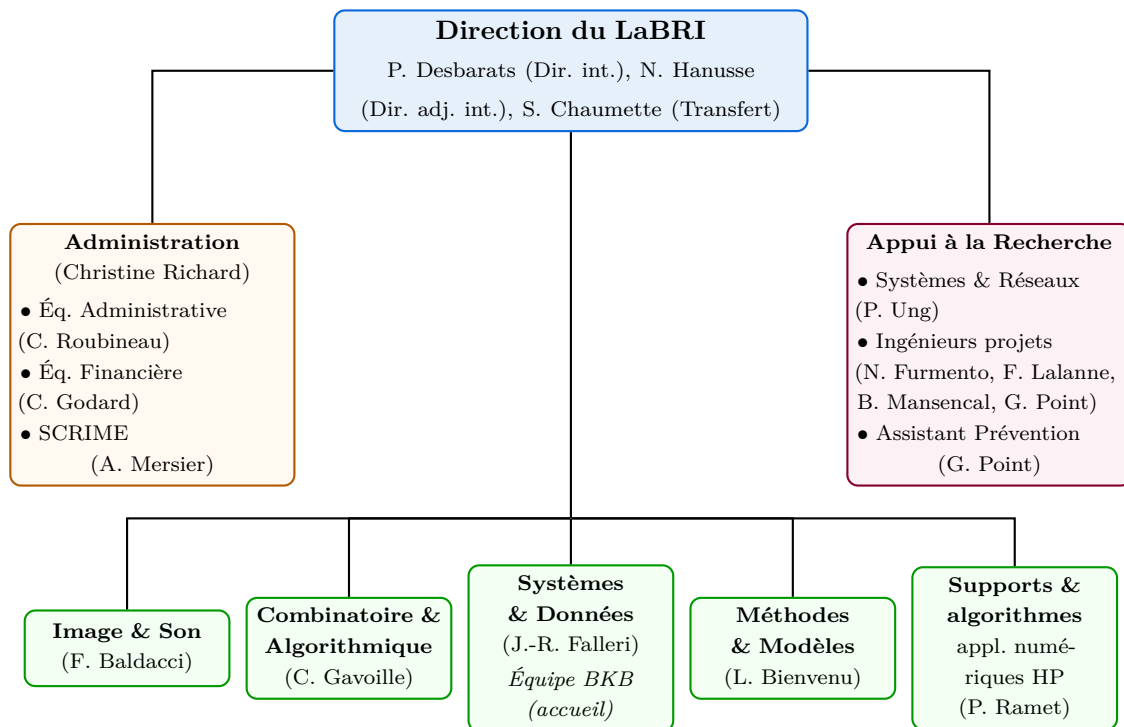


FIGURE 9 – Organigramme structurel simplifié du LaBRI.

B Détails des jeux de données

Cette section fournit d’abord les détails des huit jeux de données scRNA-seq du sous-ensemble commun (“common-8”) utilisés pour la construction et la comparaison principale, y compris l’espèce, les tailles (cellules et gènes), le nombre de batchs, le nombre de populations cellulaires rares, ainsi que leur niveau d’effet de batch (mesuré par PCR sur PCA) et de déséquilibre de classe (mesuré par l’indice de Gini). Elle décrit ensuite les deux jeux supplémentaires réservés à l’évaluation de scRAW sur des données non vues pendant l’optimisation des hyperparamètres.

Dataset	Espèce	Cellules	Gènes	Batchs	Rares (< 5%)	PCR/PCA	Gini
BBAG094 Zeisel	Souris	3 006	19 972	10	4	0,499	0,543
BBAG094 spleen	Souris	9 552	23 341	2	3	0,022	0,644
Baron Human Pancreas	Humain	8 569	20 125	4	9	0,048	0,644
Human testis GSE112013	Humain	6 490	27 477	6	4	0,025	0,304
Kang PBMC	Humain	24 679	35 635	16	1	0,069	0,500
Macaque retina bipolar	Macaque	30 302	36 162	30	4	0,184	0,388
Paul15 bone marrow	Souris	2 730	3 451	1	9	0,000	0,410
Tabula Muris liver	Souris	2 859	22 966	1	5	0,000	0,578

TABLE 6 – Récapitulatif des huit jeux de données du sous-ensemble commun (common-8) utilisés dans nos expériences.

Description des métriques de caractérisation des jeux de données

- **PCR sur PCA (Principal Component Regression on PCA)** : cette métrique estime la part de variance portée par le batch dans l’espace PCA. Elle est inspirée de scIB, qui utilise la régression sur composantes principales pour quantifier la contribution d’une covariable comme le batch [38].
- **Indice de Gini des classes** : cette métrique résume le déséquilibre des effectifs entre types cellulaires. Un score proche de 0 indique des classes de tailles comparables, tandis qu’un score élevé indique quelques classes dominantes et des classes minoritaires. L’usage du coefficient de Gini pour détecter un déséquilibre multi-classe est par exemple employé par Hirsch et al. [42].

Provenance et traitement des jeux de données

Dataset	Origine
BBAG094 Zeisel	Collection BBAG094 associée au dépôt GitHub/article scSCCNIA [43]; source Zeisel listée dans l'import local depuis Hemberg Lab.
BBAG094 spleen	Collection BBAG094 associée au dépôt GitHub/article scSCCNIA [43]; fichier Spleen issu du dépôt GitHub <code>xuebaliang/scziDesk</code> , cité dans la disponibilité des données de l'article.
Baron Human Pancreas	Données Baron et al. 2016 / GSE84133, avec quatre donneurs humains.
Human testis GSE112013	Archive GEO GSE112013 : table UMI et matrice de métadonnées.
Kang PBMC	Archive GEO GSE96583 : matrices MTX, table de gènes et métadonnées batch2.
Macaque retina bipolar	Données GSE118480 reprises depuis l'archive de reproduction scDML.
Paul15 bone marrow	Chargeur natif <code>scanpy.datasets.paul15()</code> .
Tabula Muris liver	Sous-ensemble foie de Tabula Muris, technologie FACS/Smart-seq2.

TABLE 7 – Résumé synthétique de l'origine des huit jeux de données common-8.

Jeux de données non vus pendant l'optimisation

Dataset	Espèce	Cellules	Gènes	Batches	Rares	U.-rares	PCR/PCA	Gini
Human Pancreas	Humain	6 339	14 813	4	7	6	0,156	0,651
SCIB Mouse Pancreas 1	Souris	822	14 878	1	9	5	0,000	0,711

TABLE 8 – Caractéristiques des deux jeux de données utilisés pour évaluer scRAW sur des données non vues pendant l'optimisation des hyperparamètres.

Dataset	Origine
Human Pancreas	Jeu <code>openproblems_v1/pancreas</code> d'OpenProblems, dérivé des benchmarks scIB [38]. La source regroupe des cellules pancréatiques humaines issues de plusieurs technologies.
SCIB Mouse Pancreas 1	Jeu pancréas souris préparé depuis le fichier source mouse1 (<code>GSM2230761</code>) de GSE84133_RAW, associé aux données Baron [33].

TABLE 9 – Origine des deux jeux de données de validation externe. Pour le jeu de données Human Pancreas, seuls les batchs `celseq`, `celseq2`, `fluidigm1` et `smartseq2` ont été conservés, tandis que les batchs communs avec Baron Human Pancreas (`inDrop`) et ceux issus de SMARTER-seq ont été retirés.

C Tableau des métriques d'évaluation

Cette section regroupe les définitions, formules mathématiques et interprétations des métriques utilisées pour évaluer et comparer les performances des algorithmes de clustering, tant sur l'aspect global que sur la détection des classes rares et ultra-rares.

Métrique	Rôle	Formule ou principe	Interprétation
ARI	Clustering global	$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]}$, où RI mesure l'accord entre paires de cellules.	Mesure si deux cellules regroupées ensemble dans la vérité biologique le sont aussi dans la prédiction, en corrigeant l'accord attendu au hasard.
NMI	Clustering global	$NMI(Y, \hat{Y}) = \frac{2I(Y; \hat{Y})}{H(Y) + H(\hat{Y})}$	Mesure la quantité d'information partagée entre les annotations biologiques et les groupes prédits.
ACC	Classification après appariement des clusters	$ACC = \frac{1}{n} \max_{\pi} \sum_{i=1}^n \mathbf{1}\{y_i = \pi(\hat{y}_i)\}$	Donne la proportion globale de cellules correctement attribuées après correspondance entre clusters et classes.
BalancedACC	Classification équilibrée	$BalancedACC = \frac{1}{ C } \sum_{c \in C} \frac{TP_c}{N_c}$	Calcule la moyenne des rappels par classe, afin qu'une classe très abondante ne masque pas les erreurs sur les classes petites.
RareACC	Classification des classes rares	$RareACC = \frac{\sum_{c \in \mathcal{R}} TP_c}{\sum_{c \in \mathcal{R}} N_c}$, avec $\mathcal{R} = \{c \mid N_c/n < 5\%\}$	Calcule la proportion brute de cellules correctement affectées parmi l'ensemble des populations rares.
UltraRareACC	Classification des classes ultra-rares	$UltraRareACC = \frac{\sum_{c \in \mathcal{U}} TP_c}{\sum_{c \in \mathcal{U}} N_c}$, avec $\mathcal{U} = \{c \mid N_c/n < 1\%\}$	Calcule la proportion brute de cellules correctement affectées parmi l'ensemble des populations ultra-rares.
Temps	Coût computationnel	$t_{exec} = t_{fin} - t_{debut}$	Permet de comparer les méthodes non seulement sur leur qualité de clustering, mais aussi sur leur coût pratique.

TABLE 10 – Métriques utilisées pour comparer les méthodes. TP_c correspond au nombre de cellules de la classe c correctement attribuées après appariement, et N_c au nombre total de cellules de cette classe.

Dans les formules ci-dessus, y_i désigne l'annotation biologique de la cellule i , $\hat{y}_i$ le cluster prédit par la méthode, n le nombre total de cellules, $\mathcal{C}$ l'ensemble des classes biologiques et π l'appariement optimal obtenu par l'algorithme hongrois.

D Résultats complets sur les huit jeux de données de l'annexe B

Famille utilisée dans les figures	Algorithmes associés
Rare Specific	scAIDE, scCAD, GiniClust, DeepScena et CellSIUS.
Méthodes généralistes	PCA+Leiden, scVI, scMAE et scNAME.
Correction batch	Harmony, ComBat, DESC et Scanorama.

TABLE 11 – Synthèse des familles de méthodes utilisées pour organiser les résultats de la section scRAW. scVI est conservé dans la famille des méthodes généralistes pour les figures de synthèse, même si le modèle peut intégrer une information de batch.

Résultats complets par famille sur les 8 jeux de données communs

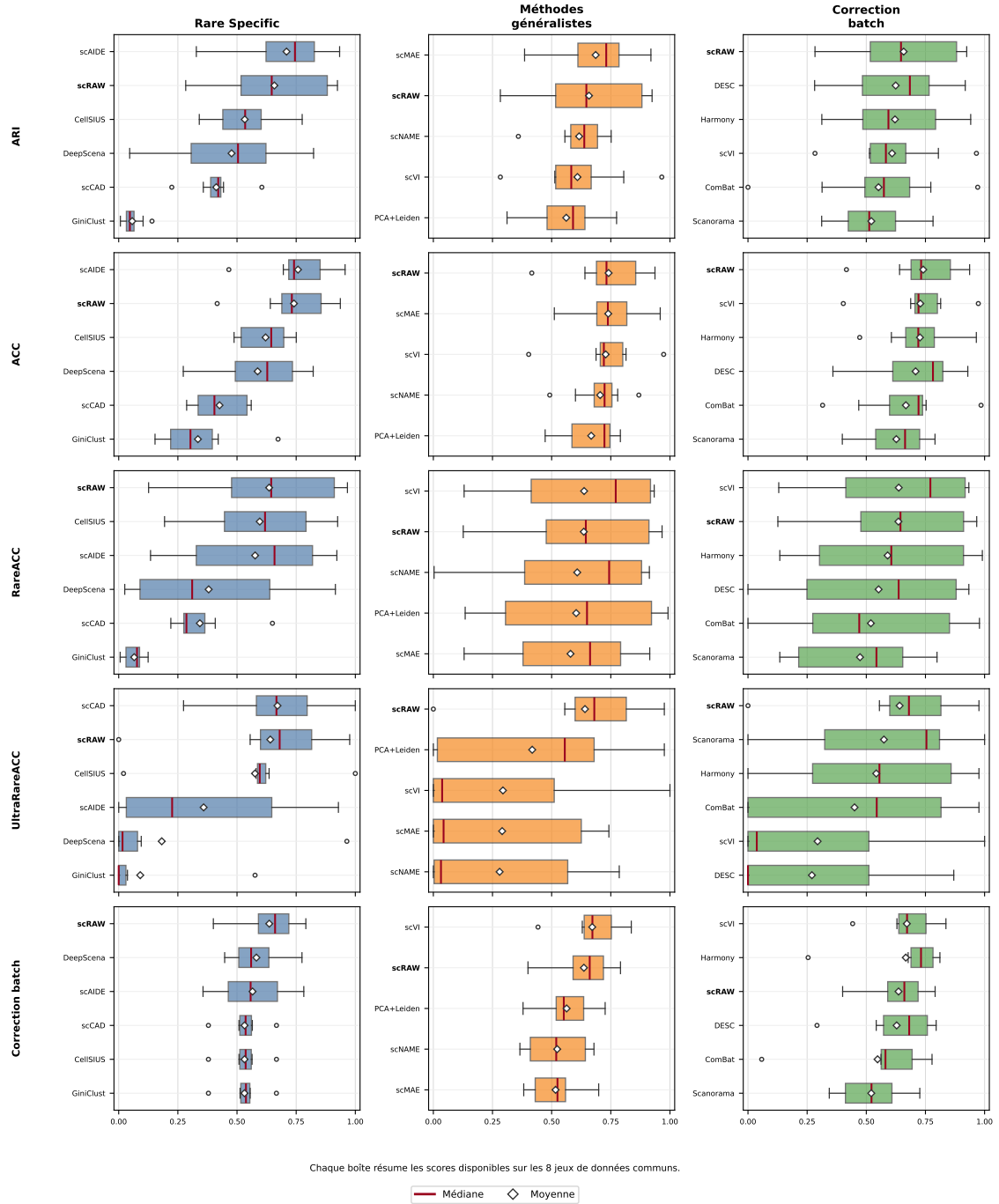


FIGURE 10 – Résultats complets par famille sur les huit jeux de données communs. Les boîtes représentent la distribution des scores disponibles par méthode sur les jeux de données ; pour l'UltraRareACC, les jeux sans classe ultra-rare ne contribuent pas à la distribution. Les méthodes sont ordonnées par moyenne croissante dans chaque sous-panneau. **scRAW** est ajouté comme repère dans chaque famille.

Les tableaux suivants présentent les résultats bruts détaillés par jeu de données pour chaque métrique d'évaluation pour les méthodes principales :

Jeu de données	scRAW	Rare Specific					Généralistes				Correction Batch			
	scRAW	scAIDE	CellSIUS	DeepScena	scCAD	GiniClust	scVI	scMAE	PCA+Leiden	scNAME	Harmony	ComBat	DESC	Scanorama
Zeisel	0,430	<u>0,696</u>	0,572	0,316	0,358	0,036	0,513	0,692	0,632	0,645	0,388	0,000	0,752	0,632
Spleen	0,870	0,934	0,465	0,517	0,416	0,141	<u>0,572</u>	<u>0,920</u>	0,433	0,689	0,518	0,580	0,336	0,448
Baron Pancreas	<u>0,916</u>	0,794	0,666	0,572	0,605	0,052	0,621	0,770	0,666	0,708	0,851	0,653	0,918	0,621
Human Testis	0,611	0,611	0,581	0,492	0,399	0,043	0,594	0,651	0,626	0,631	<u>0,652</u>	0,569	0,668	0,471
Kang PBMC	0,546	0,811	0,775	0,773	0,428	0,052	<u>0,805</u>	0,779	0,774	0,751	<u>0,774</u>	0,773	0,533	0,782
Macaque Retina	0,924	0,873	0,497	0,046	0,425	0,023	<u>0,965</u>	0,801	0,497	0,556	0,941	0,971	0,805	0,351
Bone Marrow	0,283	0,328	0,341	0,277	0,224	0,007	0,283	0,386	0,311	<u>0,359</u>	0,312	0,313	0,282	0,311
TM Liver	0,683	0,627	0,365	0,824	0,443	0,103	0,519	0,491	0,554	0,589	0,535	0,554	<u>0,700</u>	0,554
Moyenne	0,658	0,709	0,533	0,477	0,412	0,057	0,609	<u>0,686</u>	0,562	0,616	0,621	0,552	0,624	0,521

TABLE 12 – Résultats bruts de Rand Index Ajusté (ARI) pour les méthodes principales sur les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare Specific					Généralistes				Correction Batch			
	scRAW	scAIDE	CellSIUS	DeepScena	scCAD	GiniClust	scVI	scMAE	PCA+Leiden	scNAME	Harmony	ComBat	DESC	Scanorama
Zeisel	0,641	0,835	0,625	0,552	0,343	0,297	0,716	0,832	0,790	0,779	0,606	0,315	<u>0,834</u>	0,790
Spleen	0,829	<u>0,957</u>	0,687	0,729	0,557	0,673	0,796	0,959	0,606	0,869	0,687	0,733	0,518	0,641
Baron Pancreas	0,934	0,750	0,731	0,660	0,537	0,310	0,712	0,731	0,727	0,739	0,891	0,722	<u>0,929</u>	0,687
Human Testis	0,718	0,726	0,664	0,596	0,315	0,217	0,725	0,720	0,744	0,709	0,730	0,642	0,820	0,562
Kang PBMC	0,706	0,732	0,750	0,749	0,408	0,421	0,814	0,744	<u>0,754</u>	0,747	0,753	0,753	0,643	0,744
Macaque Retina	0,937	0,900	0,524	0,273	0,560	0,220	<u>0,973</u>	0,813	0,525	0,601	0,965	0,985	0,806	0,399
Bone Marrow	0,416	0,466	0,488	0,316	0,287	0,153	0,403	0,511	0,473	<u>0,492</u>	0,472	0,468	0,359	0,473
TM Liver	0,746	0,696	0,497	0,822	0,401	0,387	0,688	0,603	0,719	0,706	0,709	0,719	<u>0,758</u>	0,719
Moyenne	<u>0,741</u>	0,758	0,621	0,587	0,426	0,335	0,728	0,739	0,667	0,705	0,727	0,667	0,708	0,627

TABLE 13 – Résultats bruts de Précision Globale (ACC) pour les méthodes principales sur les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare Specific					Généralistes				Correction Batch			
	scRAW	scAIDE	CellSIUS	DeepScena	scCAD	GiniClust	scVI	scMAE	PCA+Leiden	scNAME	Harmony	ComBat	DESC	Scanorama
Zeisel	0,611	<u>0,811</u>	0,486	0,735	0,281	0,124	0,627	0,773	0,486	0,876	0,400	0,000	0,497	0,486
Spleen	0,529	0,682	0,694	0,916	0,265	0,079	0,933	0,709	0,992	0,913	<u>0,990</u>	0,978	0,933	0,799
Baron Pancreas	<u>0,896</u>	0,635	0,785	0,606	0,349	0,076	0,918	0,616	0,812	0,893	0,811	0,812	0,776	0,738
Human Testis	0,967	0,844	0,813	0,358	0,291	0,110	0,915	0,847	<u>0,921</u>	0,835	0,906	0,613	0,865	0,599
Kang PBMC	0,678	0,220	0,194	0,263	0,220	0,006	0,158	0,216	<u>0,331</u>	0,003	0,328	0,328	0,289	0,178
Macaque Retina	0,958	0,922	0,925	0,026	0,650	0,038	0,918	0,915	0,925	0,651	0,925	0,969	0,925	0,626
Bone Marrow	0,126	0,364	0,329	0,111	0,279	0,009	0,499	<u>0,434</u>	0,227	0,425	0,227	0,320	0,131	0,227
TM Liver	0,323	0,135	0,543	0,027	<u>0,408</u>	0,081	0,130	<u>0,130</u>	0,135	0,269	0,135	0,135	0,000	0,135
Moyenne	<u>0,636</u>	0,577	0,596	0,380	0,343	0,065	0,637	0,580	0,604	0,608	0,590	0,519	0,552	0,473

TABLE 14 – Résultats bruts de Précision sur Classes Rares (RareACC) pour les méthodes principales sur les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare Specific					Généralistes				Correction Batch			
	scRAW	scAIDE	CellSIUS	DeepScena	scCAD	GiniClust	scVI	scMAE	PCA+Leiden	scNAME	Harmony	ComBat	DESC	Scanorama
Zeisel	0,556	0,667	0,593	0,000	0,593	0,037	0,556	0,556	0,556	0,704	0,556	0,000	0,556	0,556
Spleen	0,976	0,000	1,000	0,000	1,000	0,024	1,000	0,000	0,976	0,786	0,976	0,976	0,000	1,000
Baron Pancreas	0,814	0,627	0,636	0,000	0,915	0,576	0,466	0,695	0,797	0,432	0,788	0,788	0,466	0,754
Human Testis	0,643	0,929	0,607	0,964	0,571	0,000	0,000	0,000	0,000	0,000	0,929	0,000	0,000	0,857
Kang PBMC	0,817	0,058	0,596	–	0,274	0,000	0,037	0,043	0,562	0,000	0,540	0,544	0,870	0,093
Macaque Retina	0,680	0,007	0,020	0,095	0,667	0,000	0,000	0,000	0,034	0,007	0,007	0,844	0,000	0,762
Bone Marrow	0,000	0,226	0,581	0,032	0,677	0,000	0,000	0,742	0,000	0,032	0,000	0,000	0,000	0,000
TM Liver	–	–	–	–	–	–	–	–	–	–	–	–	–	–
Moyenne	<u>0,641</u>	0,359	0,576	0,182	0,671	0,091	0,294	0,291	0,418	0,280	0,542	0,450	0,270	0,575

TABLE 15 – Résultats bruts de Précision sur Classes Ultra-Rares (UltraRareACC) pour les méthodes principales sur les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare Specific					Généralistes				Correction Batch			
	scRAW	scAIDE	CellSIUS	DeepScena	scCAD	GiniClust	scVI	scMAE	PCA+Leiden	scNAME	Harmony	ComBat	DESC	Scanorama
Zeisel	0,400	0,356	0,379	0,448	0,379	0,379	0,442	0,408	0,379	0,382	0,254	0,058	0,292	0,378
Spleen	0,791	0,690	0,564	0,645	0,565	0,556	0,837	0,551	0,554	0,675	0,811	0,778	0,542	0,514
Baron Pancreas	0,575	0,614	0,550	0,602	0,549	0,550	0,630	0,498	0,548	0,493	0,737	0,566	0,697	0,530
Human Testis	0,730	0,782	0,667	0,775	0,667	0,667	0,775	0,699	0,662	0,679	0,797	0,562	0,778	0,634
Kang PBMC	0,684	0,501	0,523	0,504	0,523	0,526	0,662	0,561	0,726	0,545	0,726	0,726	0,795	0,726
Macaque Retina	0,637	0,451	0,508	0,517	0,508	0,513	0,683	0,382	0,510	0,366	0,677	0,596	0,666	0,344
Bone Marrow	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
TM Liver	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
Moyenne	0,636	0,566	0,532	0,582	0,532	0,532	0,671	0,517	0,563	0,523	0,667	0,548	0,628	0,521

TABLE 16 – Résultats bruts de Correction de Batch pour les méthodes principales sur les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée. Les jeux sans effet de batch exploitable sont indiqués NA et exclus de la moyenne.

Pour faciliter la lecture des résultats, chaque sous-panneau de la figure 4 affiche **scRAW** ainsi que les trois meilleures méthodes de la famille pour la métrique considérée (sur la base de la moyenne des huit jeux de données). Les méthodes restantes, ainsi que des résultats complémentaires avec les méthodes de correction de batch associées aux différents algorithmes, sont fournis dans cette annexe. Nous ne présentons pas dans le panneau principal les méthodes couplées à un outil de correction de batch, car les auteurs ne décrivent pas cet usage, ni comme prétraitement recommandé, ni comme composant du modèle. Ces variantes sont donc conservées uniquement comme analyses complémentaires. Pour les méthodes de deep learning généralistes, Harmony a été appliqué sur l’espace latent appris, avant le clustering final, car il s’agissait de la correction de batch la plus robuste dans les expériences de référence [44]. Cette pratique est justifiée par la conception même de Harmony : l’algorithme opère sur tout espace de faible dimension et ne suppose pas que celui-ci provienne d’une PCA obligatoirement. En revanche, aucun des papiers originaux des méthodes benchmarkées ici (scDeepCluster, scMAE, scNAME, scCDG) ne propose ni n’évalue ce couplage de manière systématique : notre protocole comble ce manque en évaluant l’apport de Harmony sur l’espace latent de plusieurs architectures profondes avec des métriques globales et centrées sur les populations rares.

Résultats complémentaires avec Harmony

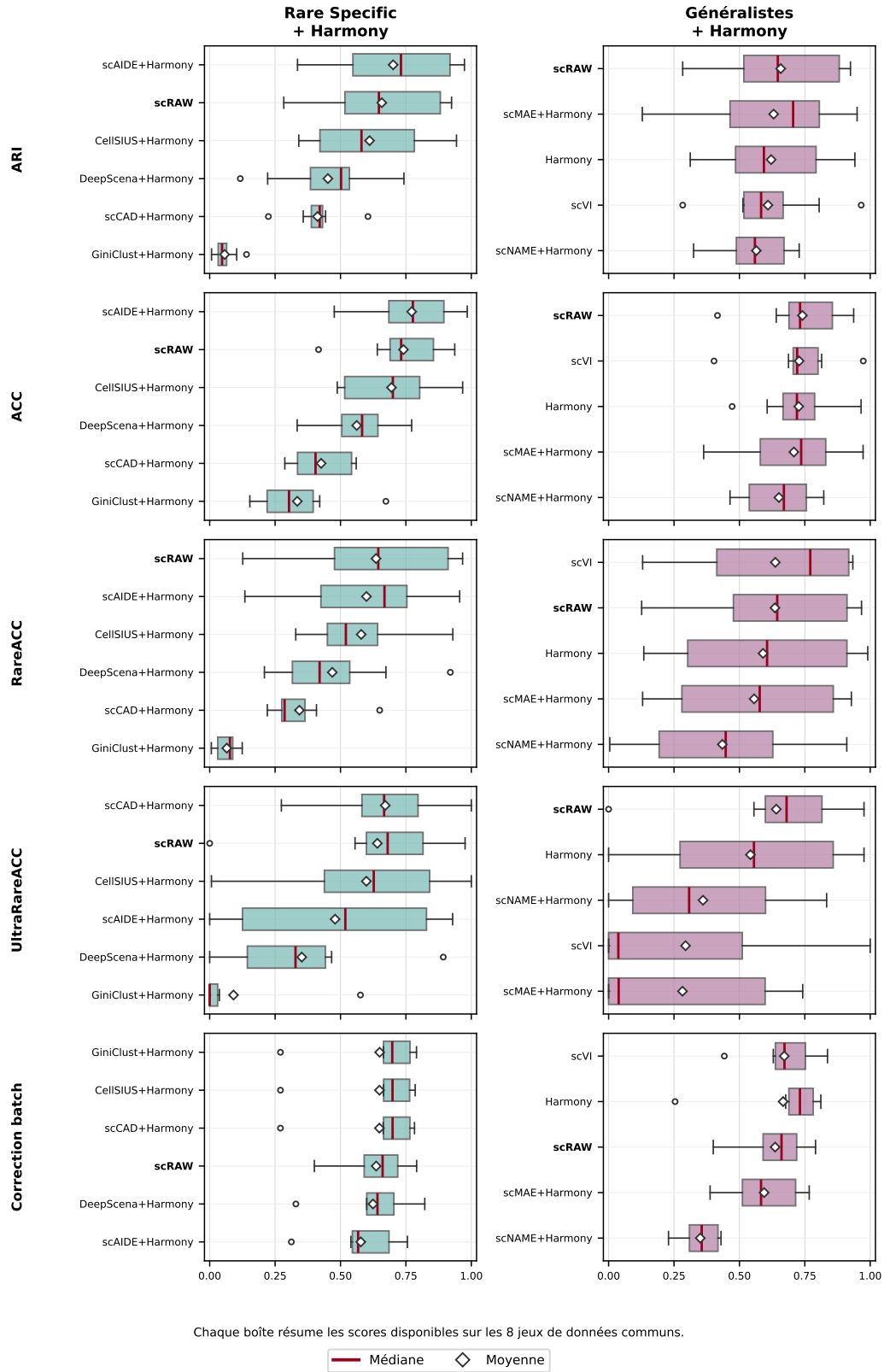


FIGURE 11 – Résultats complémentaires avec Harmony. Les méthodes *Rare Specific*+Harmony sont indiquées à titre exploratoire, car cette association n'est pas documentée comme usage standard par les auteurs. Pour les méthodes de deep learning généralistes, Harmony est appliqué sur l'espace latent avant le clustering final. La ligne Harmony correspond au pipeline PCA+Harmony avant clustering ; scVI est conservé comme méthode native intégrant l'information de batch.

Les tableaux suivants présentent les résultats bruts détaillés par jeu de données pour chaque métrique d'évaluation pour les expériences complémentaires avec Harmony :

Jeu de données	scRAW	Rare + Harmony					Généralistes + Harmony			
	scRAW	scAIDE	CellSIUS	scCAD	DeepScena	GiniClust	Harmony	scVI	scMAE	scNAME
Zeisel	0,430	0,394	0,440	0,358	0,117	0,036	0,388	<u>0,513</u>	0,129	0,722
Spleen	0,870	0,931	0,558	0,416	0,487	0,141	0,518	<u>0,572</u>	<u>0,911</u>	0,621
Baron Pancreas	0,916	<u>0,914</u>	0,895	0,605	0,440	0,052	0,851	0,621	<u>0,770</u>	0,728
Human Testis	0,611	<u>0,599</u>	0,603	0,399	0,523	0,043	<u>0,652</u>	0,594	0,651	0,653
Kang PBMC	0,546	0,816	0,745	0,428	0,742	0,052	0,774	<u>0,805</u>	0,760	0,492
Macaque Retina	0,924	0,974	0,943	0,425	0,518	0,023	0,941	<u>0,965</u>	0,950	0,498
Bone Marrow	0,283	0,336	<u>0,341</u>	0,224	0,222	0,007	0,312	0,283	0,386	0,325
TM Liver	0,683	<u>0,647</u>	0,365	0,443	0,566	0,103	0,535	0,519	0,491	0,477
Moyenne	<u>0,658</u>	0,701	0,611	0,412	0,452	0,057	0,621	0,609	0,631	0,565

TABLE 17 – Résultats bruts complémentaires de Rand Index Ajusté (ARI) avec Harmony pour les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare + Harmony					Généralistes + Harmony			
	scRAW	scAIDE	CellSIUS	scCAD	DeepScena	GiniClust	Harmony	scVI	scMAE	scNAME
Zeisel	0,641	0,625	0,523	0,343	0,363	0,297	0,606	<u>0,716</u>	0,363	0,743
Spleen	0,829	0,959	0,763	0,557	0,679	0,673	0,687	0,796	<u>0,955</u>	0,822
Baron Pancreas	0,934	0,874	<u>0,922</u>	0,537	0,552	0,310	0,891	0,712	0,788	0,738
Human Testis	0,718	0,705	0,669	0,315	0,595	0,217	<u>0,730</u>	0,725	0,724	0,796
Kang PBMC	0,706	0,833	0,731	0,408	0,772	0,421	0,753	<u>0,814</u>	0,748	0,490
Macaque Retina	0,937	0,984	0,967	0,560	0,570	0,220	0,965	<u>0,973</u>	0,973	0,553
Bone Marrow	0,416	0,476	<u>0,488</u>	0,287	0,334	0,153	0,472	0,403	0,511	0,464
TM Liver	0,746	<u>0,720</u>	0,497	0,401	0,631	0,387	0,709	0,688	0,603	0,602
Moyenne	<u>0,741</u>	0,772	0,695	0,426	0,562	0,335	0,727	0,728	0,708	0,651

TABLE 18 – Résultats bruts complémentaires de Précision Globale (ACC) avec Harmony pour les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare + Harmony					Généralistes + Harmony			
	scRAW	scAIDE	CellSIUS	scCAD	DeepScena	GiniClust	Harmony	scVI	scMAE	scNAME
Zeisel	0,611	0,649	0,454	0,281	0,324	0,124	0,400	<u>0,627</u>	0,319	0,032
Spleen	0,529	0,734	0,562	0,265	0,920	0,079	0,990	<u>0,933</u>	0,721	0,910
Baron Pancreas	0,896	0,688	0,880	0,349	0,674	0,076	0,811	<u>0,918</u>	0,928	0,561
Human Testis	0,967	0,814	0,498	0,291	0,489	0,110	0,906	<u>0,915</u>	0,841	0,827
Kang PBMC	0,678	0,441	0,438	0,220	<u>0,488</u>	0,006	0,328	0,158	0,164	0,247
Macaque Retina	0,958	<u>0,955</u>	0,929	0,650	<u>0,352</u>	0,038	0,925	0,918	0,913	0,438
Bone Marrow	0,126	<u>0,379</u>	0,329	0,279	0,209	0,009	0,227	0,499	0,434	<u>0,458</u>
TM Liver	0,323	0,135	0,543	<u>0,408</u>	0,291	0,081	0,135	0,130	0,130	0,004
Moyenne	<u>0,636</u>	0,599	0,579	0,343	0,468	0,065	0,590	0,637	0,556	0,435

TABLE 19 – Résultats bruts complémentaires de Précision sur Classes Rares (RareACC) avec Harmony pour les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare + Harmony					Généralistes + Harmony			
	scRAW	scAIDE	CellSIUS	scCAD	DeepScena	GiniClust	Harmony	scVI	scMAE	scNAME
Zeisel	<u>0,556</u>	0,519	0,296	0,593	0,370	0,037	<u>0,556</u>	<u>0,556</u>	0,519	0,185
Spleen	<u>0,976</u>	0,000	1,000	1,000	0,286	0,024	<u>0,976</u>	1,000	0,000	0,833
Baron Pancreas	<u>0,814</u>	0,780	0,627	0,915	0,466	0,576	<u>0,788</u>	0,466	0,678	0,458
Human Testis	<u>0,643</u>	0,929	<u>0,893</u>	0,571	<u>0,893</u>	0,000	0,929	0,000	0,000	0,000
Kang PBMC	0,817	0,058	<u>0,788</u>	0,274	–	0,000	0,540	0,037	0,038	0,308
Macaque Retina	<u>0,680</u>	0,878	0,007	0,667	0,000	0,000	0,007	0,000	0,000	0,000
Bone Marrow	0,000	0,194	0,581	<u>0,677</u>	0,097	0,000	0,000	0,000	0,742	0,742
TM Liver	–	–	–	–	–	–	–	–	–	–
Moyenne	<u>0,641</u>	0,479	0,599	0,671	0,352	0,091	0,542	0,294	0,282	0,361

TABLE 20 – Résultats bruts complémentaires de Précision sur Classes Ultra-Rares (UltraRareACC) avec Harmony pour les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée.

Jeu de données	scRAW	Rare + Harmony					Généralistes + Harmony			
	scRAW	scAIDE	CellSIUS	scCAD	DeepScena	GiniClust	Harmony	scVI	scMAE	scNAME
Zeisel	0,400	0,312	0,270	0,270	0,329	0,270	0,254	0,442	0,388	0,402
Spleen	0,791	0,756	0,785	0,783	<u>0,822</u>	0,790	0,811	0,837	0,767	0,310
Baron Pancreas	0,575	0,562	0,730	0,729	0,681	<u>0,730</u>	0,737	0,630	0,604	0,309
Human Testis	0,730	0,723	0,776	0,777	0,712	<u>0,777</u>	0,797	0,775	0,752	0,430
Kang PBMC	<u>0,684</u>	0,573	0,664	0,663	0,602	0,664	0,726	0,662	0,561	0,423
Macaque Retina	0,637	0,540	0,668	0,669	0,600	0,666	<u>0,677</u>	0,683	0,495	0,229
Bone Marrow	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
TM Liver	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
Moyenne	0,636	0,578	0,649	0,649	0,624	0,650	<u>0,667</u>	0,671	0,594	0,350

TABLE 21 – Résultats bruts complémentaires de Correction de Batch avec Harmony pour les 8 jeux de données communs. La meilleure performance par jeu de données est en gras, et la deuxième est soulignée. Les jeux sans effet de batch exploitable sont indiqués NA et exclus de la moyenne.

Ces résultats détaillés permettent d’analyser le comportement individuel de chaque méthode sur chacun des jeux de données, offrant ainsi une vision plus fine et contrastée que les seules moyennes globales.

E Résultats complets du transfert de loss pondérée $\mathcal{L}_{\text{recon,ponderee}}$

Cette annexe présente les résultats complets de l’expérience d’intégration de la loss pondérée $\mathcal{L}_{\text{recon,ponderee}}$ de **scRAW** dans **scMAE**, **scDeepCluster** et **DESC**. Les valeurs correspondent à la phase complète, avec cinq seeds par configuration.

La baseline correspond à l’algorithme sans pondération ajoutée. Les autres lignes correspondent aux variantes où une pondération rare-aware est ajoutée à la loss de reconstruction $\mathcal{L}_{\text{recon}}$, ce qui produit une version $\mathcal{L}_{\text{recon,ponderee}}$. La pondération est activée après une phase de warm-up, à l’époque 55, comme dans la configuration par défaut de **scRAW**. Les variantes avec triplet loss $\mathcal{L}_{\text{triplet}}$ activent cette composante à partir de l’époque 60.

Le Tableau 22 présente la comparaison synthétique entre chaque baseline et sa meilleure variante pondérée.

Algorithme	Configuration	ARI	RareACC	UltraRareACC
scMAE	Baseline	0,665	0,562	0,492
	Meilleure pondérée : Pseudo clustering Leiden	0,648	0,589	0,518
scDeepCluster	Baseline	0,422	0,622	0,222
	Meilleure pondérée : Pseudo clustering Leiden	0,451	0,645	0,344
DESC	Baseline	0,585	0,631	0,429
	Meilleure pondérée : Pseudo clustering KMeans + triplet	0,576	0,662	0,414

TABLE 22 – Comparaison entre chaque algorithme de base et sa meilleure variante avec loss pondérée **scRAW**. Les valeurs sont des moyennes par jeu de données ; la meilleure variante est sélectionnée par la moyenne des trois métriques.

Pour résumer les performances, les seeds sont d’abord moyennées pour chaque jeu de données, puis les jeux de données sont moyennés avec le même poids. Le score utilisé pour sélectionner la meilleure configuration non-baseline est la moyenne de ARI, RareACC et UltraRareACC. L’UltraRareACC a été reprise des fichiers de résultats détaillés lorsqu’elle était disponible, ou recalculée à partir des rappels par type cellulaire lorsque seule la colonne **ClassWise** était exportée.

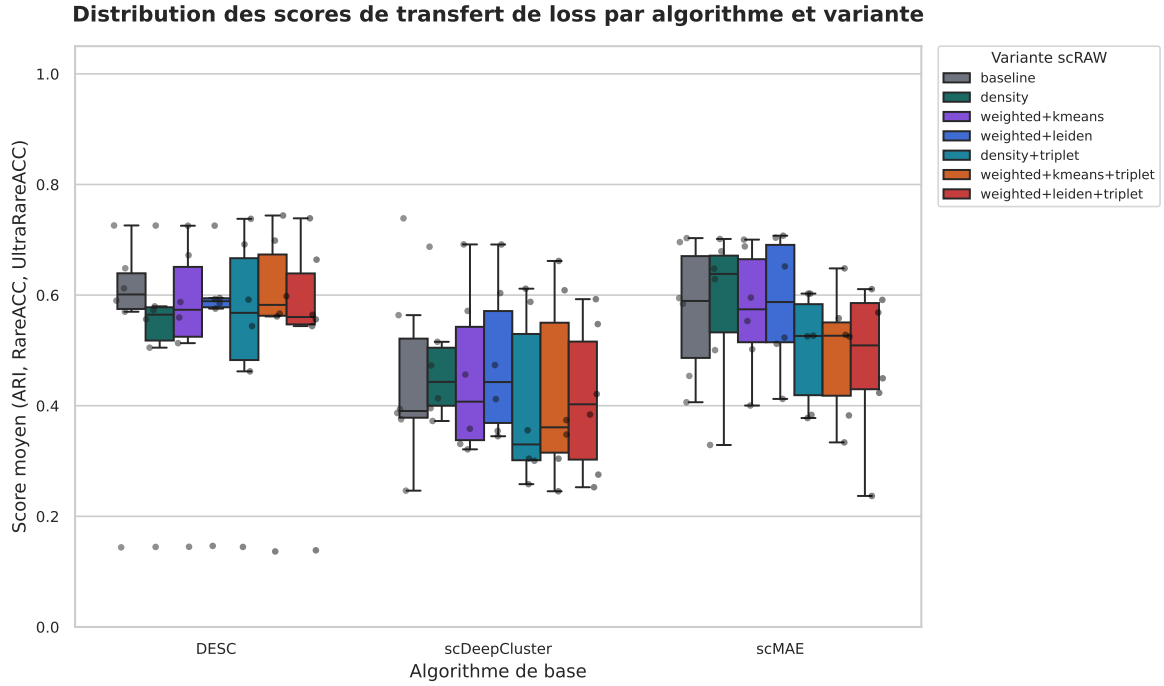


FIGURE 12 – Distribution des scores pour toutes les configurations testées lors du transfert de la loss pondérée $\mathcal{L}_{\text{recon, ponderee}}$ de scRAW. Les boîtes représentent la distribution des scores moyens (moyenne de l'ARI, de la RareACC et de l'UltraRareACC) sur l'ensemble des jeux de données.

Algorithme	Configuration	ARI	RareACC	UltraRareACC	Score
scMAE	Baseline	0,665	0,562	0,492	0,573
	Pondération densité	0,636	0,581	0,527	0,581
	Pondération complète Leiden	0,648	0,589	0,518	0,585
	Pondération complète KMeans	0,612	0,603	0,505	0,573
	Pondération complète Leiden + triplet	0,483	0,640	0,317	0,480
	Pondération complète KMeans + triplet	0,549	0,614	0,325	0,496
	Pondération densité + triplet KMeans	0,552	0,615	0,343	0,503
scDeepCluster	Baseline	0,422	0,622	0,222	0,422
	Pondération densité	0,471	0,644	0,313	0,476
	Pondération complète Leiden	0,451	0,645	0,344	0,480
	Pondération complète KMeans	0,461	0,619	0,284	0,455
	Pondération complète Leiden + triplet	0,484	0,464	0,289	0,412
	Pondération complète KMeans + triplet	0,498	0,508	0,264	0,424
	Pondération densité + triplet KMeans	0,502	0,510	0,197	0,403
DESC	Baseline	0,585	0,631	0,429	0,548
	Pondération densité	0,592	0,596	0,355	0,514
	Pondération complète Leiden	0,593	0,606	0,411	0,537
	Pondération complète KMeans	0,595	0,635	0,371	0,534
	Pondération complète Leiden + triplet	0,580	0,633	0,389	0,534
	Pondération complète KMeans + triplet	0,576	0,662	0,414	0,551
	Pondération densité + triplet KMeans	0,573	0,630	0,383	0,529

TABLE 23 – Résultats moyens de toutes les configurations expérimentées pour l'intégration de la loss pondérée scRAW. Le score correspond à la moyenne de ARI, RareACC et UltraRareACC ; la meilleure configuration non-baseline de chaque algorithme est en gras.

F Détails de l'évaluation inductive

Cette annexe détaille les résultats utilisés pour la figure inductive synthétique du rapport. Les valeurs correspondent aux moyennes par jeu de données sur les splits inductifs disponibles.

Jeu de données	Groupes d'entraînement	Groupe(s) test non vus
Baron pancreas	Trois donneurs parmi <code>human1</code> , <code>human2</code> , <code>human3</code> , <code>human4</code>	Un donneur laissé de côté; quatre splits au total
BBAG094 spleen	Donneur 3-F-56	Donneur 3-M-8
Human testis	Réplicats des donneurs 1 et 2	Deux réplicats du donneur 3, évalués séparément
Kang PBMC	Tous les donneurs sauf le donneur test	Donneurs 1039 et 107, évalués séparément
Macaque retina	Individus M1 et M2	Individus M3 et M4, évalués séparément
Human Pancreas	Technologies <code>smartseq2</code> et <code>celseq2</code>	Technologies <code>celseq</code> et <code>fluidigm1</code> , évaluées séparément

TABLE 24 – Définition des splits inductifs utilisés dans la comparaison. Chaque split simule l'application d'un modèle entraîné sur des groupes connus à un groupe non observé pendant l'entraînement.

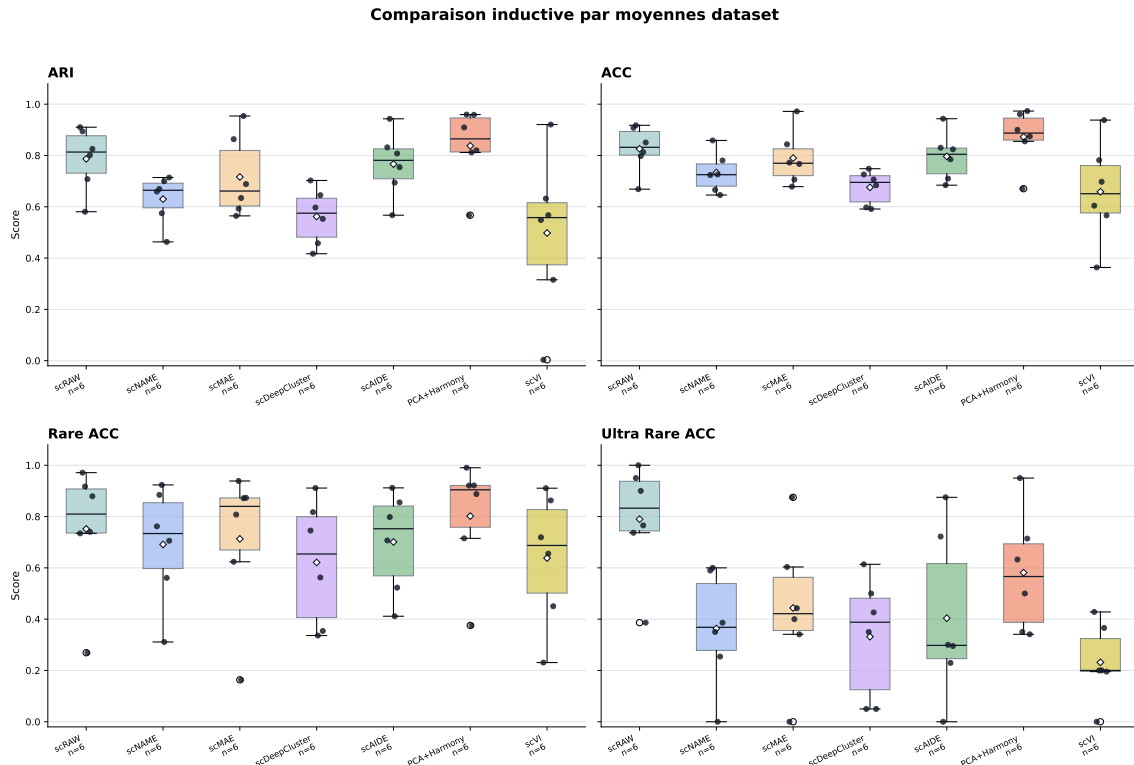


FIGURE 13 – Distribution des performances inductives pour ARI, ACC, RareACC et Ultra-RareACC. Les boîtes représentent la distribution des moyennes par jeu de données et split inductif pour chaque algorithme testé. Chaque point noir correspond à la moyenne d'un jeu de données et les losanges blancs indiquent la moyenne globale de l'algorithme.

Métrique	Jeu de données	scRAW	scNAME	scMAE	scDeepCluster	scAIDE	PCA+Harmony	scVI
ARI	Baron pancreas	0,910	0,659	0,864	0,703	0,832	0,909	0,548
	BBAG094 spleen	0,894	0,669	0,565	0,417	0,943	0,959	0,003
	Human testis	0,581	0,575	0,593	0,597	0,567	0,567	0,632
	Kang PBMC	0,708	0,699	0,634	0,458	0,694	0,812	0,567
	Macaque retina	0,801	0,463	0,954	0,645	0,808	0,959	0,921
	Pancreas 4 batches	0,826	0,714	0,688	0,553	0,754	0,820	0,315
ACC	Baron pancreas	0,908	0,724	0,844	0,726	0,830	0,900	0,604
	BBAG094 spleen	0,918	0,859	0,772	0,598	0,943	0,961	0,363
	Human testis	0,669	0,666	0,678	0,748	0,684	0,670	0,782
	Kang PBMC	0,798	0,726	0,706	0,591	0,710	0,855	0,697
	Macaque retina	0,813	0,646	0,972	0,706	0,785	0,973	0,938
	Pancreas 4 batches	0,851	0,781	0,767	0,684	0,824	0,875	0,566
Rare ACC	Baron pancreas	0,734	0,762	0,624	0,745	0,707	0,715	0,719
	BBAG094 spleen	0,971	0,885	0,808	0,817	0,798	0,990	0,231
	Human testis	0,917	0,923	0,939	0,911	0,912	0,921	0,911
	Kang PBMC	0,269	0,311	0,163	0,336	0,411	0,375	0,656
	Macaque retina	0,740	0,561	0,872	0,563	0,523	0,921	0,863
	Pancreas 4 batches	0,879	0,706	0,873	0,354	0,855	0,888	0,450
Ultra Rare ACC	Baron pancreas	0,737	0,590	0,603	0,614	0,722	0,632	0,428
	BBAG094 spleen	0,950	0,600	0,000	0,050	0,000	0,950	0,000
	Human testis	1,000	0,350	0,875	0,350	0,875	0,350	0,200
	Kang PBMC	0,900	0,000	0,400	0,500	0,300	0,500	0,200
	Macaque retina	0,765	0,254	0,443	0,426	0,230	0,714	0,366
	Pancreas 4 batches	0,386	0,386	0,341	0,050	0,295	0,341	0,195

TABLE 25 – Moyennes par jeu de données des expériences inductives pour les quatre métriques principales. Chaque valeur correspond à la moyenne des splits inductifs disponibles pour un couple jeu de données–algorithme ; la meilleure valeur de chaque ligne est en gras.

G Étude d'ablation de scRAW

Cette section présente l'étude d'ablation menée pour valider l'apport individuel de chaque composante de **scRAW** : la pondération dynamique $\mathcal{L}_{\text{recon,ponderee}}$, la triplet loss rare-aware $\mathcal{L}_{\text{triplet}}$ et la correction adversariale de batch DANN $\mathcal{L}_{\text{batch}}$. Toutes les variantes ont été entraînées sur le jeu de données Baron Human Pancreas en utilisant HDBSCAN comme algorithme de clustering final.

Le Tableau 26 détaille les scores obtenus pour les variantes suivantes :

- **MSE** : Autoencodeur classique avec une loss MSE standard $\mathcal{L}_{\text{recon}}$, sans pondération, sans triplet loss $\mathcal{L}_{\text{triplet}}$ et sans correction adversariale $\mathcal{L}_{\text{batch}}$.
- **Weighted MSE** : Variante incluant la pondération dynamique des cellules $\mathcal{L}_{\text{recon,ponderee}}$ par taille de pseudo-cluster et densité locale, mais sans triplet loss $\mathcal{L}_{\text{triplet}}$ ni correction adversariale $\mathcal{L}_{\text{batch}}$.
- **Weighted + triplet** : Autoencodeur pondéré avec triplet loss rare-aware $\mathcal{L}_{\text{triplet}}$, sans correction de batch adversariale $\mathcal{L}_{\text{batch}}$.
- **Weighted + DANN** : Autoencodeur pondéré avec correction adversariale (DANN) $\mathcal{L}_{\text{batch}}$, sans triplet loss $\mathcal{L}_{\text{triplet}}$.
- **Full (scRAW)** : Modèle complet associant la pondération dynamique $\mathcal{L}_{\text{recon,ponderee}}$, la triplet loss $\mathcal{L}_{\text{triplet}}$ et la branche adversariale $\mathcal{L}_{\text{batch}}$.
- **Full single update** : Modèle scRAW complet, mais où la mise à jour des poids cellulaires n'est effectuée qu'une seule fois au lieu d'être actualisée périodiquement.
- **Full density-only** : Modèle scRAW complet, mais où la pondération des cellules repose uniquement sur la densité locale (la composante liée à la taille des pseudo-clusters est désactivée).
- **Full cluster-size-only** : Modèle scRAW complet, mais où la pondération repose uniquement sur la taille des pseudo-clusters Leiden (la composante liée à la densité locale est désactivée).

Variante	ARI	NMI	ACC	B. ACC	Rare	U. Rare	scIB-E	Temps (s)
MSE	0,189	0,397	0,407	0,427	0,350	0,695	0,427	70,8
Weighted MSE	0,269	0,455	0,456	0,520	0,390	0,686	0,426	518,2
Weighted + triplet	0,851	0,871	0,885	0,813	0,809	0,805	0,431	454,9
Weighted + DANN	0,246	0,560	0,545	0,611	0,491	0,653	0,574	279,7
Full (scRAW)	0,916	0,884	0,934	0,870	0,896	0,814	0,590	337,9
Full single update	0,900	0,873	0,913	0,747	0,793	0,686	0,599	285,9
Full density-only	0,920	0,888	0,926	0,753	0,796	0,703	0,587	334,4
Full cluster-size-only	0,723	0,777	0,800	0,869	0,897	0,949	0,596	347,2

TABLE 26 – Résultats de l’étude d’ablation des composantes de scRAW sur le jeu de données Baron Human Pancreas (clustering final HDBSCAN). B. ACC : BalancedACC; Rare : RareACC; U. Rare : UltraRareACC; scIB-E : score d’intégration de batch scIB (entropie de mélange).

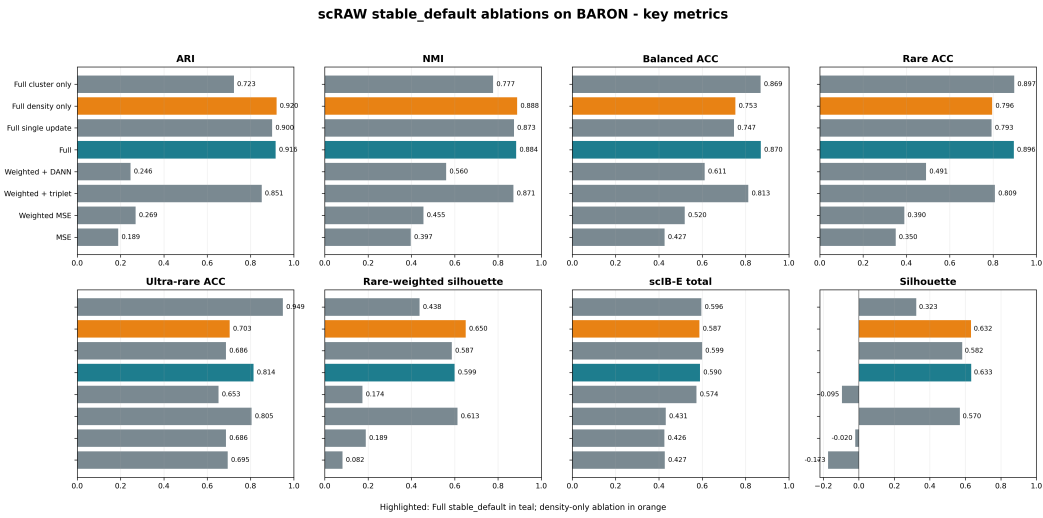


FIGURE 14 – Comparaison graphique des performances des variantes d’ablation de scRAW pour les métriques de clustering global (ARI, NMI, ACC), la détection des classes rares (RareACC, UltraRareACC) et le score d’intégration de batch (scIB-E).

H Hyperparamètres de la configuration de référence scRAW

Cette annexe détaille les hyperparamètres effectivement actifs dans la configuration par défaut utilisée pour décrire scRAW dans le rapport.

TABLE 27 – Hyperparamètres actifs du preset `stable_default` de scRAW.

Bloc	Paramètre	Valeur	Rôle
Prétraitement	<code>n_top_genes</code>	2000	Nombre de gènes hautement variables conservés.
Prétraitement	<code>min_genes_per_cell</code>	200	Filtrage des cellules de mauvaise qualité.
Prétraitement	<code>min_cells_per_gene</code>	3	Filtrage des gènes très peu détectés.
Prétraitement	<code>target_sum</code>	20000	Normalisation des comptes par cellule.
Prétraitement	<code>scale_max_value</code>	10.0	Seuil appliqué après standardisation.
Prétraitement	<code>hvg_flavor</code>	<code>seurat</code>	Méthode de sélection des HVG.
Architecture	<code>hidden_layers</code>	512,256,128	Tailles des couches cachées de l'encodeur.
Architecture	<code>z_dim</code>	256	Dimension de l'espace latent.
Architecture	<code>dropout</code>	0.3	Dropout dans l'encodeur et le décodeur.
Entraînement	<code>epochs</code>	120	Nombre total d'époques.
Entraînement	<code>warmup_epochs</code>	55	Phase sans pondération dynamique.
Entraînement	<code>lr</code>	0.00164076083297036	Taux d'apprentissage initial.
Entraînement	<code>batch_size</code>	192	Taille des mini-batches.
Entraînement	<code>seed</code>	64	Graine utilisée pour l'entraînement.
Reconstruction	<code>reconstruction_distribution</code>	<code>mse</code>	Loss de reconstruction $\mathcal{L}_{\text{recon}}$ utilisée.
Reconstruction	<code>masking_rate</code>	0.1	Fraction d'entrées masquées aléatoirement.
Reconstruction	<code>masking_value</code>	0.0	Valeur utilisée pour les entrées masquées.
Reconstruction	<code>masked_recon_weight</code>	0.8	Poids interne des positions masquées.
Reconstruction	<code>masking_apply_weighted</code>	<code>True</code>	Masquage aussi actif pendant la phase pondérée.
Pondération	<code>weight_exponent</code>	0.2	Intensité de la pondération par taille de pseudo-cluster.
Pondération	<code>density_knn_k</code>	15	Voisinage utilisé pour estimer la densité locale.
Pondération	<code>density_weight_exponent</code>	1.0	Exposant de la composante densité.
Pondération	<code>density_weight_clip</code>	3.0	Borne supérieure avant normalisation de la densité.
Pondération	<code>weight_fusion_mode</code>	<code>multiplicative</code>	Fusion des composantes cluster et densité.
Pondération	<code>cluster_weight_power</code>	1.0	Puissance appliquée à la composante cluster.
Pondération	<code>density_weight_power</code>	1.0	Puissance appliquée à la composante densité.
Pondération	<code>dynamic_weight_momentum</code>	0.6884621079434989	Lissage des poids entre deux mises à jour.
Pondération	<code>dynamic_weight_update_interval</code>	20	Fréquence de recalcul des poids.

Suite page suivante

Bloc	Paramètre	Valeur	Rôle
Pondération	min_cell_weight	0.3845423008053828	Borne inférieure du poids cellulaire final.
Pondération	max_cell_weight	10.0	Borne supérieure du poids cellulaire final.
Pseudo-labels	pseudo_label_method	leiden	Méthode utilisée pour les pseudo-clusters.
Pseudo-labels	pseudo_leiden_resolution_strategy	gamma_default	Recherche de résolution Leiden utilisée par scRAW.
Pseudo-labels	unsupervised_k_selection	heuristic	Estimation heuristique de K si nécessaire.
Pseudo-labels	unsupervised_k_min	8	Borne minimale de K .
Pseudo-labels	unsupervised_k_max	30	Borne maximale de K .
Triplet	rare_triplet_weight	0.05007581780188212	Poids de la triplet loss $\mathcal{L}_{\text{triplet}}$.
Triplet	rare_triplet_start_epoch	60	Début de la triplet loss $\mathcal{L}_{\text{triplet}}$.
Triplet	rare_triplet_margin	0.4	Marge de séparation.
Triplet	rare_triplet_min_weight	1.2	Poids minimal pour choisir une ancre.
Triplet	max_triplet_anchors_per_batch	64	Nombre maximal d'ancres par mini-batch.
Batch	use_batch_conditioning	True	Activation de la branche batch si plusieurs modalités de batch sont exploitables.
Batch	batch_correction_key	batch	Colonne de métadonnées utilisée pour le batch.
Batch	adversarial_batch_weight	0.11763398875166495	Poids de la loss adversariale batch $\mathcal{L}_{\text{batch}}$.
Batch	adversarial_lambda	1.0	Intensité de l'inversion de gradient.
Batch	adversarial_start_epoch	30	Début de la branche adversariale.
Batch	adversarial_ramp_epochs	30	Durée de montée progressive du signal adversarial.
Clustering final	hdbscan_min_cluster_size	8	Taille minimale d'un cluster HDBSCAN.
Clustering final	hdbscan_min_samples	6	Nombre minimal d'échantillons HDBSCAN.
Clustering final	hdbscan_cluster_selection_method	leiden	Stratégie de sélection des clusters.
Clustering final	hdbscan_reassign_noise	False	Pas de réassignation des points bruit.

I Figures de recherche d'hyperparamètres scRAW

Cette annexe regroupe les sorties Optuna d'importance moyenne des hyperparamètres pour les trois recherches utilisées dans la construction de la version par défaut de scRAW

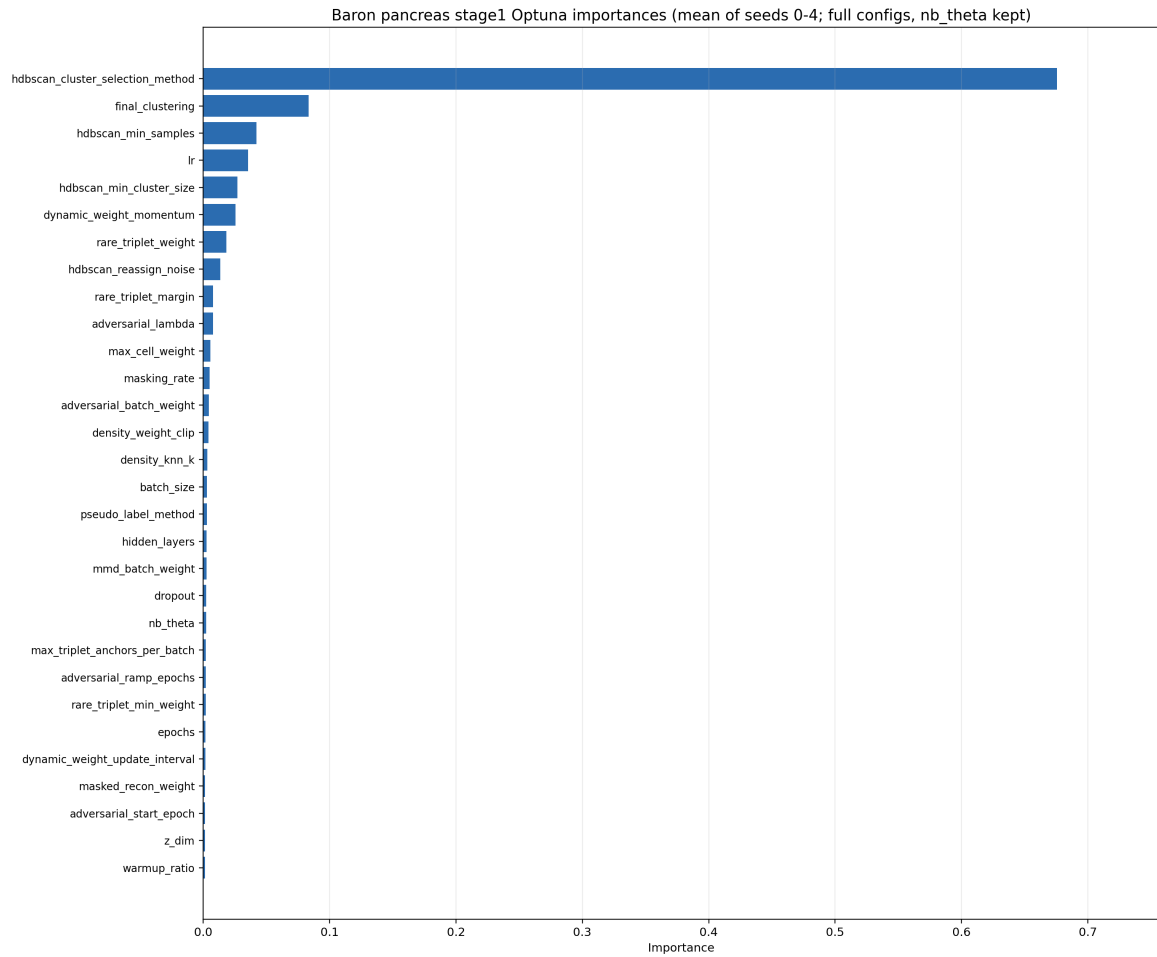


FIGURE 15 – Importance des hyperparamètres pour la recherche large sur Baron Human Pancreas.

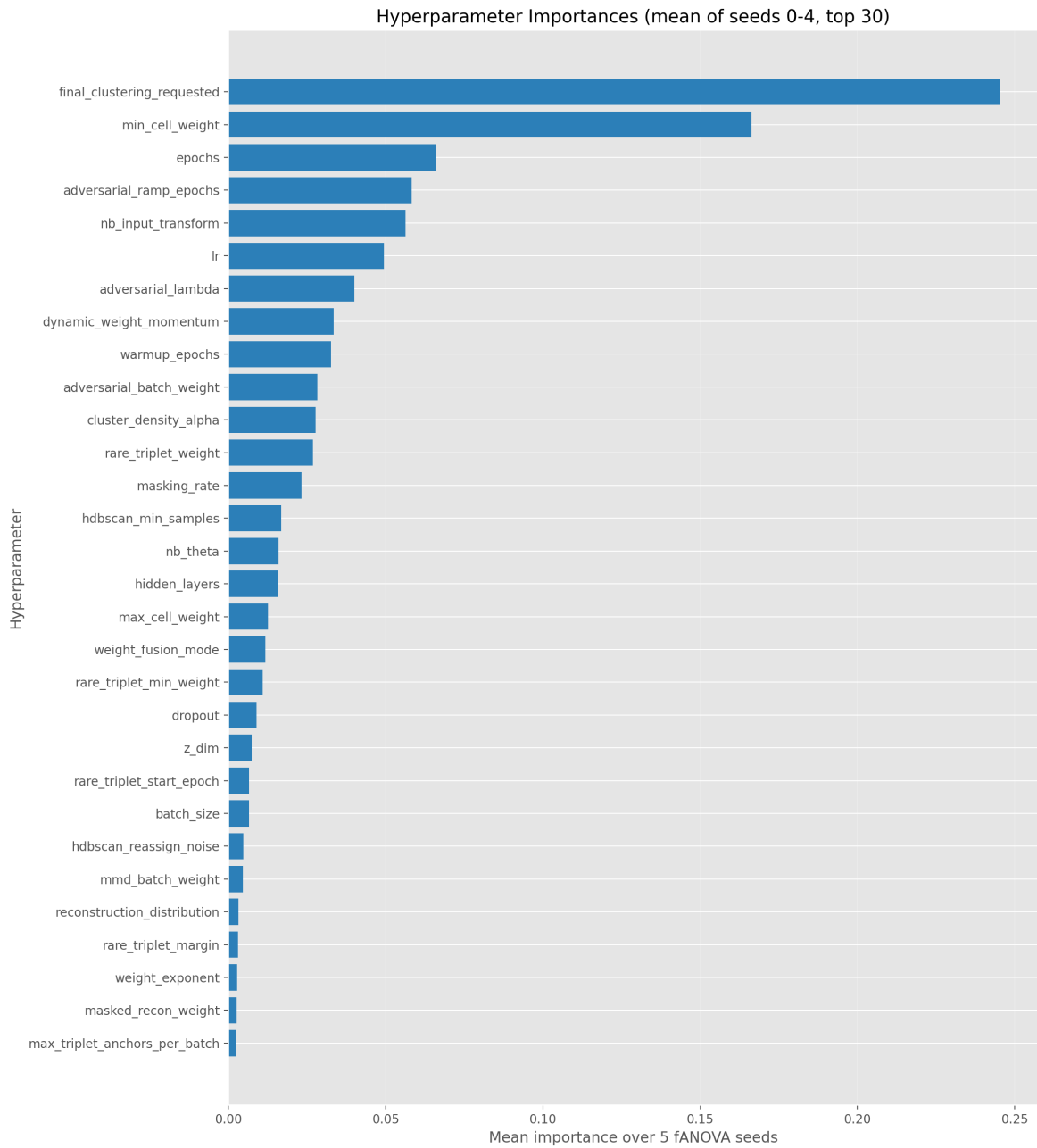


FIGURE 16 – Importance des hyperparamètres pour la recherche généraliste sur huit jeux de données.

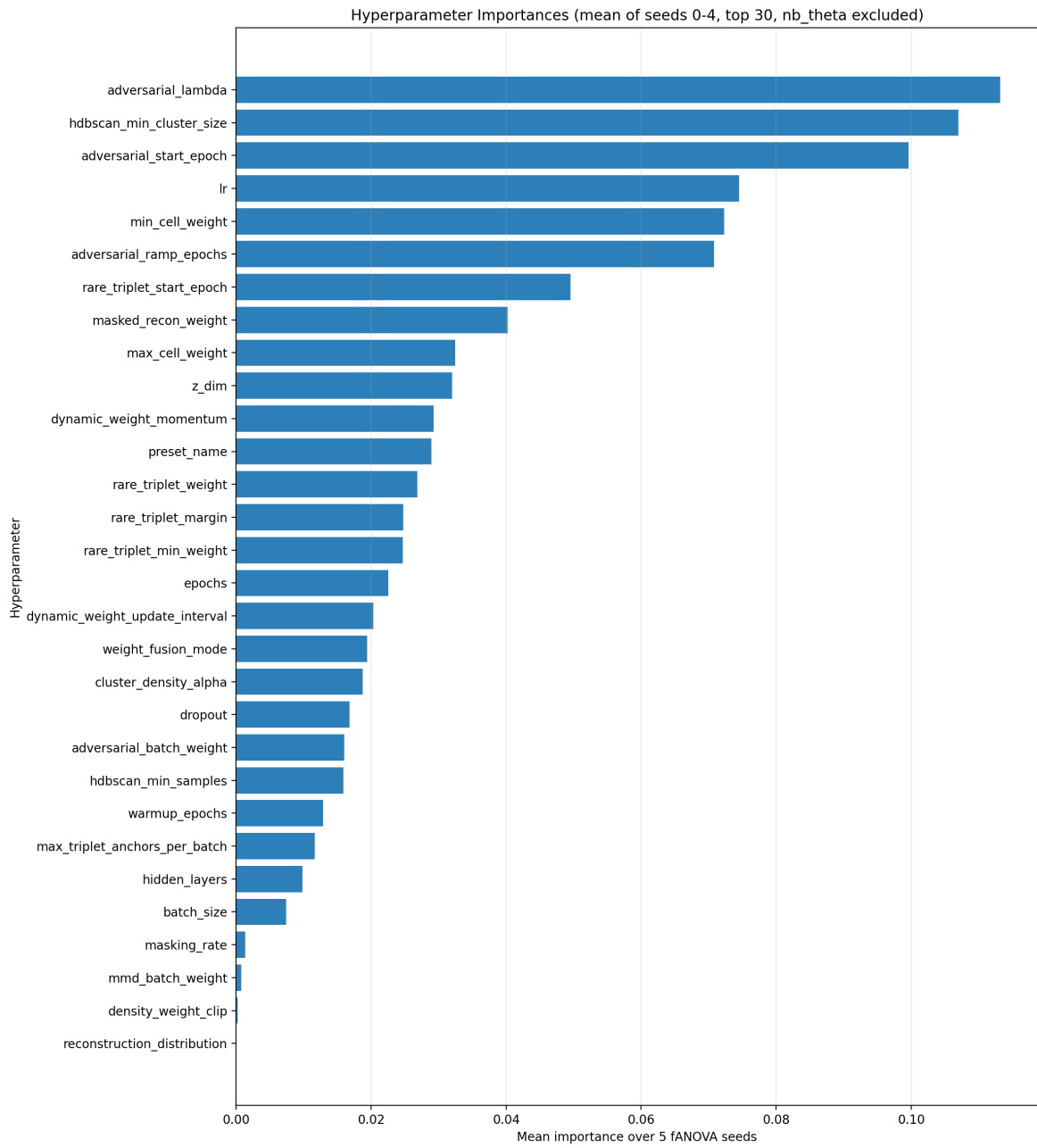


FIGURE 17 – Importance des hyperparamètres pour la recherche restreinte `stable_generalist`.

J Screenshots de l'interface graphique SCRBenchmark

Cette section regroupe les captures d'écran illustrant le fonctionnement et le flux complet de l'interface utilisateur Streamlit développée pour SCRBenchmark.

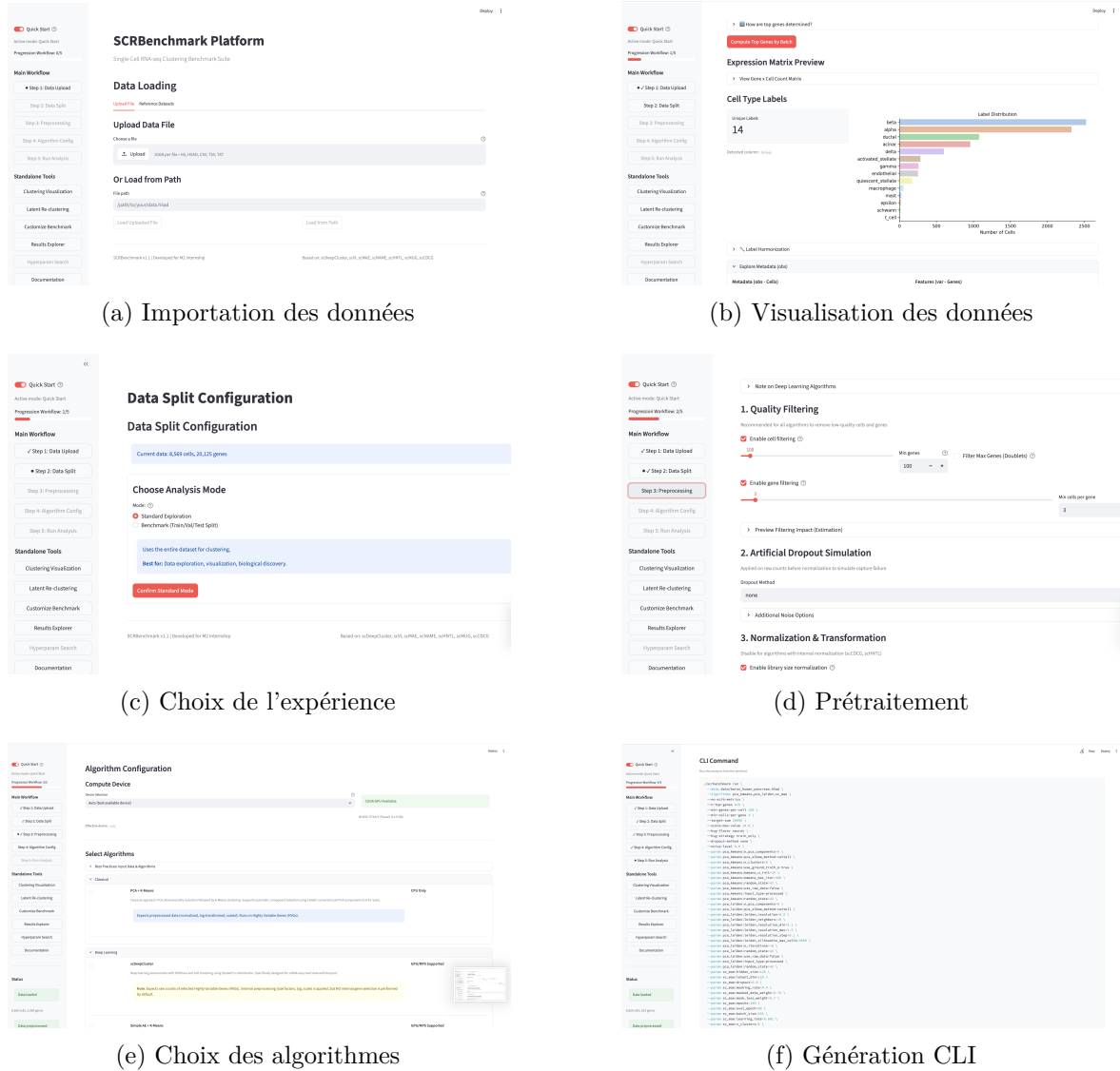
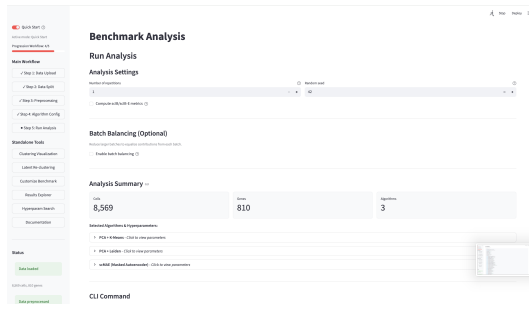


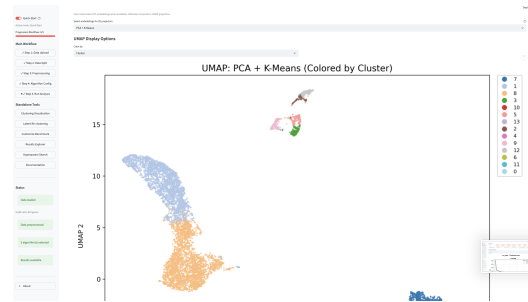
FIGURE 18 – Aperçu des différentes étapes de l'interface Streamlit de SCRBenchmark (1^{re} partie : configuration et préparation).



(g) Exécution des calculs



(h) Résultats des métriques



(i) UMAPs des résultats

FIGURE 18 – Aperçu des différentes étapes de l'interface Streamlit de SCRBenchmark (2^e partie : exécution et résultats).

Bibliographie

- [1] Dominic GRÜN et al. « Single-cell messenger RNA sequencing reveals rare intestinal cell types ». In : *Nature* 525.7568 (2015), p. 251-255. DOI : [10.1038/nature14966](https://doi.org/10.1038/nature14966).
- [2] Li LI et al. « Single-Cell RNA-Seq Analysis Maps Development of Human Germline Cells and Gonadal Niche Interactions ». In : *Cell Stem Cell* 20.6 (2017), 858-873.e4. DOI : [10.1016/j.stem.2017.03.007](https://doi.org/10.1016/j.stem.2017.03.007).
- [3] Yijie ZHANG et al. « Single-cell RNA sequencing in cancer research ». In : *Journal of Experimental & Clinical Cancer Research* 40.1 (2021), p. 81. DOI : [10.1186/s13046-021-01874-1](https://doi.org/10.1186/s13046-021-01874-1).
- [4] Atlas M. SARDOO et al. « Decoding brain memory formation by single-cell RNA sequencing ». In : *Briefings in Bioinformatics* 23.6 (2022), bbac412. DOI : [10.1093/bib/bbac412](https://doi.org/10.1093/bib/bbac412).
- [5] Hansruedi MATHYS et al. « Single-cell transcriptomic analysis of Alzheimer’s disease ». In : *Nature* 570.7761 (2019), p. 332-337. DOI : [10.1038/s41586-019-1195-2](https://doi.org/10.1038/s41586-019-1195-2).
- [6] Malte D LUECKEN et Fabian J THEIS. « Current best practices in single-cell RNA-seq analysis : a tutorial ». In : *Molecular systems biology* 15.6 (2019), e8746.
- [7] F. Alexander WOLF, Philip ANGERER et Fabian J. THEIS. « Scanpy : large-scale single-cell gene expression data analysis ». In : *Genome Biology* 19.1 (2018), p. 15. DOI : [10.1186/s13059-017-1382-0](https://doi.org/10.1186/s13059-017-1382-0).
- [8] Vladimir Yu KISELEV, Tallulah S ANDREWS et Martin HEMBERG. « Challenges in unsupervised clustering of single-cell RNA-seq data ». In : *Nature Reviews Genetics* 20.5 (2019), p. 273-282. DOI : [10.1038/s41576-018-0088-9](https://doi.org/10.1038/s41576-018-0088-9).
- [9] Romain LOPEZ et al. « Deep generative modeling for single-cell transcriptomics ». In : *Nature Methods* 15.12 (2018), p. 1053-1058. DOI : [10.1038/s41592-018-0229-2](https://doi.org/10.1038/s41592-018-0229-2).
- [10] James MACQUEEN. « Some methods for classification and analysis of multivariate observations ». In : *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*. T. 1. 14. Oakland, CA, USA. 1967, p. 281-297.
- [11] Vincent D BLONDEL et al. « Fast unfolding of communities in large networks ». In : *Journal of Statistical Mechanics : Theory and Experiment* 2008.10 (2008), P10008.
- [12] Vincent A. TRAAG, Ludo WALTMAN et Nees Jan van ECK. « From Louvain to Leiden : guaranteeing well-connected communities ». In : *Scientific Reports* 9.1 (2019), p. 5233. DOI : [10.1038/s41598-019-41695-z](https://doi.org/10.1038/s41598-019-41695-z).
- [13] Ricardo J. G. B. CAMPELLO, Davoud MOULAVI et Jörg SANDER. « Hierarchical Density Estimates for Data Clustering, Visualization, and Outlier Detection ». In : *ACM Transactions on Knowledge Discovery from Data* 10.1 (2015), p. 1-51. DOI : [10.1145/2733381](https://doi.org/10.1145/2733381).
- [14] Zhaoyu FANG, Ruiqing ZHENG et Min LI. « scMAE : a masked autoencoder for single-cell RNA-seq clustering ». In : *Bioinformatics* 40.1 (2024), btae020. DOI : [10.1093/bioinformatics/btae020](https://doi.org/10.1093/bioinformatics/btae020).

- [15] Xiangjie LI et al. « Deep learning enables accurate clustering with batch effect removal in single-cell RNA-seq analysis ». In : *Nature Communications* 11.1 (2020), p. 2338.
- [16] Tian TIAN et al. « Clustering single-cell RNA-seq data with a model-based deep learning approach ». In : *Nature Machine Intelligence* 1.4 (2019), p. 191-198.
- [17] Hui WAN, Liang CHEN et Minghua DENG. « scNAME : neighborhood contrastive clustering with ancillary mask estimation for scRNA-seq data ». In : *Bioinformatics* 38.6 (2022), p. 1575-1583. DOI : [10.1093/bioinformatics/btac011](https://doi.org/10.1093/bioinformatics/btac011).
- [18] Qi ZHU et al. « Discriminative Domain Adaption Network for Simultaneously Removing Batch Effects and Annotating Cell Types in Single-Cell RNA-Seq ». In : *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 21.6 (2024), p. 2543-2555. DOI : [10.1109/TCBB.2024.3487574](https://doi.org/10.1109/TCBB.2024.3487574).
- [19] Reut DANINO, Iftach NACHMAN et Roded SHARAN. « Batch correction of single-cell sequencing data via an autoencoder architecture ». In : *Bioinformatics Advances* 4.1 (2024), vbad186. DOI : [10.1093/bioadv/vbad186](https://doi.org/10.1093/bioadv/vbad186).
- [20] Karl PEARSON. « On lines and planes of closest fit to systems of points in space ». In : *Philosophical Magazine* 2.11 (1901), p. 559-572. DOI : [10.1080/14786440109462720](https://doi.org/10.1080/14786440109462720).
- [21] Ping XU et al. « scCDCG : Efficient Deep Structural Clustering for Single-Cell RNA-Seq via Deep Cut-Informed Graph Embedding ». In : *International Conference on Database Systems for Advanced Applications*. T. 14611. Lecture Notes in Computer Science. Springer, 2024, p. 195-210. DOI : [10.1007/978-981-97-5575-2_11](https://doi.org/10.1007/978-981-97-5575-2_11).
- [22] Lan JIANG et al. « GiniClust : detecting rare cell types from single-cell gene expression data with Gini index ». In : *Genome Biology* 17.1 (2016), p. 144. DOI : [10.1186/s13059-016-1010-4](https://doi.org/10.1186/s13059-016-1010-4).
- [23] Marilisa NERI et al. « CellSIUS provides sensitive and specific detection of rare cell populations from complex single cell RNA-seq data ». In : *Genome Biology* 20.142 (2019), p. 142. DOI : [10.1186/s13059-019-1748-0](https://doi.org/10.1186/s13059-019-1748-0).
- [24] J. LIU et al. « scAIDE : clustering of large-scale single-cell RNA-seq data reveals putative and rare cell types ». In : *NAR Genomics and Bioinformatics* 2.4 (2020), lqaa082. DOI : [10.1093/nargab/lqaa082](https://doi.org/10.1093/nargab/lqaa082).
- [25] Tianyuan LEI et al. « Self-supervised deep clustering of single-cell RNA-seq data to hierarchically detect rare cell populations ». In : *Briefings in Bioinformatics* 24.5 (2023), bbad335. DOI : [10.1093/bib/bbad335](https://doi.org/10.1093/bib/bbad335).
- [26] Yunpei XU et al. « scCAD : Cluster decomposition-based anomaly detection for rare cell identification in single-cell expression data ». In : *Nature Communications* 15.1 (2024), p. 7561. DOI : [10.1038/s41467-024-51891-9](https://doi.org/10.1038/s41467-024-51891-9).
- [27] W Evan JOHNSON, Cheng LI et Ariel RABINOVIC. « Adjusting batch effects in microarray expression data using empirical Bayes methods ». In : *Biostatistics* 8.1 (2007), p. 118-127.
- [28] Ilya KORSUNSKY et al. « Fast, sensitive and accurate integration of single-cell data with Harmony ». In : *Nature methods* 16.12 (2019), p. 1289-1296.

- [29] Brian HIE, Bryan D. BRYSON et Bonnie BERGER. « Efficient integration of heterogeneous single-cell transcriptomes using Scanorama ». In : *Nature Biotechnology* 37.6 (2019), p. 685-691. DOI : [10.1038/s41587-019-0113-3](https://doi.org/10.1038/s41587-019-0113-3).
- [30] Yaroslav GANIN et al. « Domain-Adversarial Training of Neural Networks ». In : *Journal of Machine Learning Research* 17.59 (2016), p. 1-35.
- [31] Songwei GE et al. « Supervised Adversarial Alignment of Single-Cell RNA-seq Data ». In : *Journal of Computational Biology* 28.5 (2021), p. 501-513. DOI : [10.1089/cmb.2020.0439](https://doi.org/10.1089/cmb.2020.0439).
- [32] Ping XU et al. « scCluBench : Comprehensive Benchmarking of Clustering Algorithms for Single-Cell RNA Sequencing ». In : *Proceedings of the AAAI Conference on Artificial Intelligence*. T. 40. 2026, p. 1364-1372.
- [33] Maayan BARON et al. « A single-cell transcriptomic map of the human and mouse pancreas reveals inter- and intra-cell population structure ». In : *Cell Systems* 3.4 (2016), 346-360.e4. DOI : [10.1016/j.cels.2016.08.011](https://doi.org/10.1016/j.cels.2016.08.011).
- [34] Junyuan XIE, Ross GIRSHICK et Ali FARHADI. « Unsupervised deep embedding for clustering analysis ». In : *International Conference on Machine Learning*. PMLR. 2016, p. 478-487.
- [35] Xifeng GUO et al. « Improved deep embedded clustering on images ». In : *Proceedings of the International Joint Conference on Neural Networks*. 2017, p. 2430-2437.
- [36] Mohammad LOTFOLLAHI et al. « Query-to-reference single-cell integration with transfer learning ». In : *Nature Biotechnology* 39.11 (2021), p. 1422-1434.
- [37] Harold W. KUHN. « The Hungarian Method for the Assignment Problem ». In : *Naval Research Logistics Quarterly* 2.1-2 (1955), p. 83-97. DOI : [10.1002/nav.3800020109](https://doi.org/10.1002/nav.3800020109).
- [38] Malte D. LUECKEN et al. « Benchmarking atlas-level data integration in single-cell genomics ». In : *Nature Methods* 19.1 (2022), p. 41-50. DOI : [10.1038/s41592-021-01336-8](https://doi.org/10.1038/s41592-021-01336-8).
- [39] Kilian Q. WEINBERGER, John BLITZER et Lawrence K. SAUL. « Distance metric learning for large margin nearest neighbor classification ». In : *Advances in Neural Information Processing Systems*. T. 18. 2005, p. 1473-1480.
- [40] Hua MENG, Chuan QIN et Zhiguo LONG. « scHNTL : single-cell RNA-seq data clustering augmented by high-order neighbors and triplet loss ». In : *Bioinformatics* 41.2 (2025), btaf044. DOI : [10.1093/bioinformatics/btaf044](https://doi.org/10.1093/bioinformatics/btaf044).
- [41] Takuya AKIBA et al. « Optuna : A Next-generation Hyperparameter Optimization Framework ». In : *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019, p. 2623-2631.
- [42] Vitali HIRSCH et al. « Exploiting domain knowledge to address class imbalance and a heterogeneous feature space in multi-class classification ». In : *The VLDB Journal* 32 (2023), p. 1037-1064. DOI : [10.1007/s00778-023-00780-6](https://doi.org/10.1007/s00778-023-00780-6).

- [43] Jing WANG et al. « scSCCNIA : similarity matrix based contrastive clustering with neighbor information aggregation for single-cell RNA sequencing data ». In : *Briefings in Bioinformatics* 27.2 (2026), bbag094. DOI : [10.1093/bib/bbag094](https://doi.org/10.1093/bib/bbag094).
- [44] Hoa Thi Nhu TRAN et al. « A benchmark of batch-effect correction methods for single-cell RNA sequencing data ». In : *Genome Biology* 21.1 (2020), p. 12. DOI : [10.1186/s13059-019-1850-9](https://doi.org/10.1186/s13059-019-1850-9).